

# **Liveness Analysis**

## Chapter 10

Text Book: Modern Compiler Implementation in C

Author: Andrew W. Appel

# Outline

- Liveness Analysis
  - **Why, How, What**
  - Dataflow Equation for Liveness
  - Solving the Equations
- More Discussions
- Interference Graphs
- Liveness in the Tiger Compiler

# Liveness Analysis - Why

## Source Code

```
package tutorial;

import com.hq.hglib.xena.datatypes.xml.XSDDataSetType;
import com.hq.hglib.xena.ref.model.Model;
import com.hq.hglib.xena.ref.model.Model_Property;
import com.hq.hglib.xena.ref.model.Resource;

public class HelloRDFWorld {

    /**
     * @param args
     */
    public static void main(String[] args) {
        Model m = ModelFactory.createDefaultModel();
        String ID = "http://example.org/test?";

        Resource r = m.createResource(ID + "?");
        Property p = m.createProperty(ID + "?");

        r.addProperty(p, "hello world", XSDDataType.STRING);

        m.write(System.out, "Turin");
    }
}
```

# IR

```

read x
t1=x>0
if_false t1 goto L1
fact=1
label L2
t2 = fact * x
fact = t2
t3 = x - 1

```

## Machine Code

[illegible]

## Problem:

- **Intermediate Representation**: can use unbounded number of temporaries.
- **Real machine**: has a bounded number of registers.

# Liveness Analysis - Why

- If temporaries **a** and **b** are never “in use” at the same time: they can fit into the same register.
- Thus, many temporaries can fit in a few registers.

***a = b + 1;***  
***return a;***

- If they don't all fit, the excess temporaries can be kept in memory.
- The compiler needs to determine which temporaries are “in use” at the same time. **How?**
- **Liveness Analysis**: determines the liveness of each **variable**.
  - A variable is **live** if it holds a value that may be needed in the future.

# Liveness Analysis - How

- How to perform the liveness analysis?
- => Will variable  $x$  be used after statement  $n$ ?

Which statements may be executed after  $n$ ?

➡ Make a control flow graph

Is  $x$  used in these statements?

➡ Analyze the statement.

- Backward Analysis: from the future to the past

```
 $a \leftarrow 0$   
 $L_1 : b \leftarrow a + 1$   
 $c \leftarrow c + b$   
 $a \leftarrow b * 2$   
if  $a < N$  goto  $L_1$   
return  $c$ 
```

# Liveness Analysis - Example

Make a control flow graph:

- **node**: a statement.
- **edge**: control flow. if statement **m** can be followed by statement **n**, **m**  $\rightarrow$  **n**

```

$$a \leftarrow 0$$

$$L_1 : b \leftarrow a + 1$$

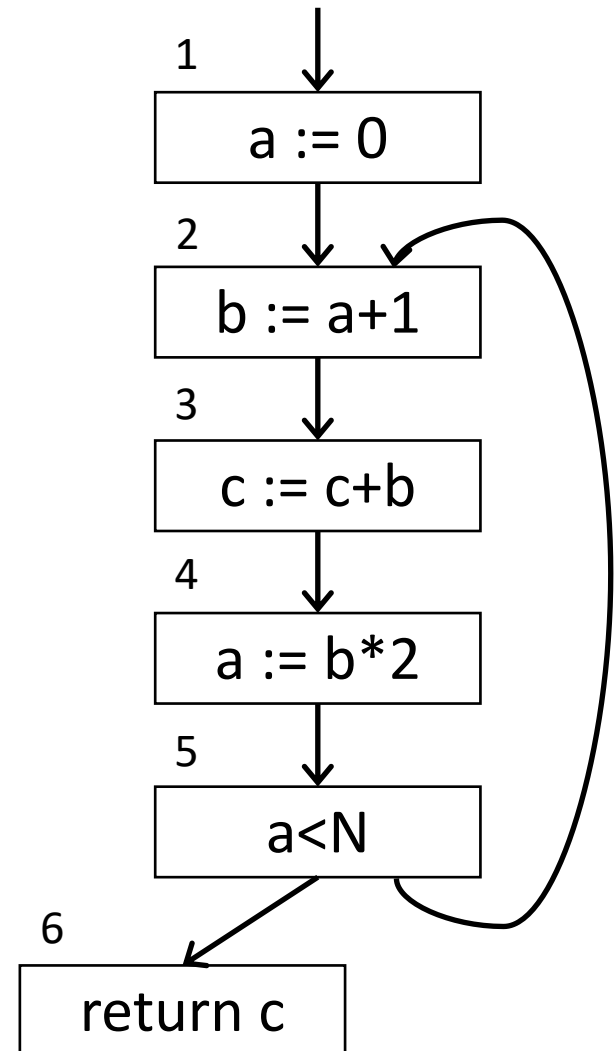
$$c \leftarrow c + b$$

$$a \leftarrow b * 2$$

$$\text{if } a < N \text{ goto } L_1$$

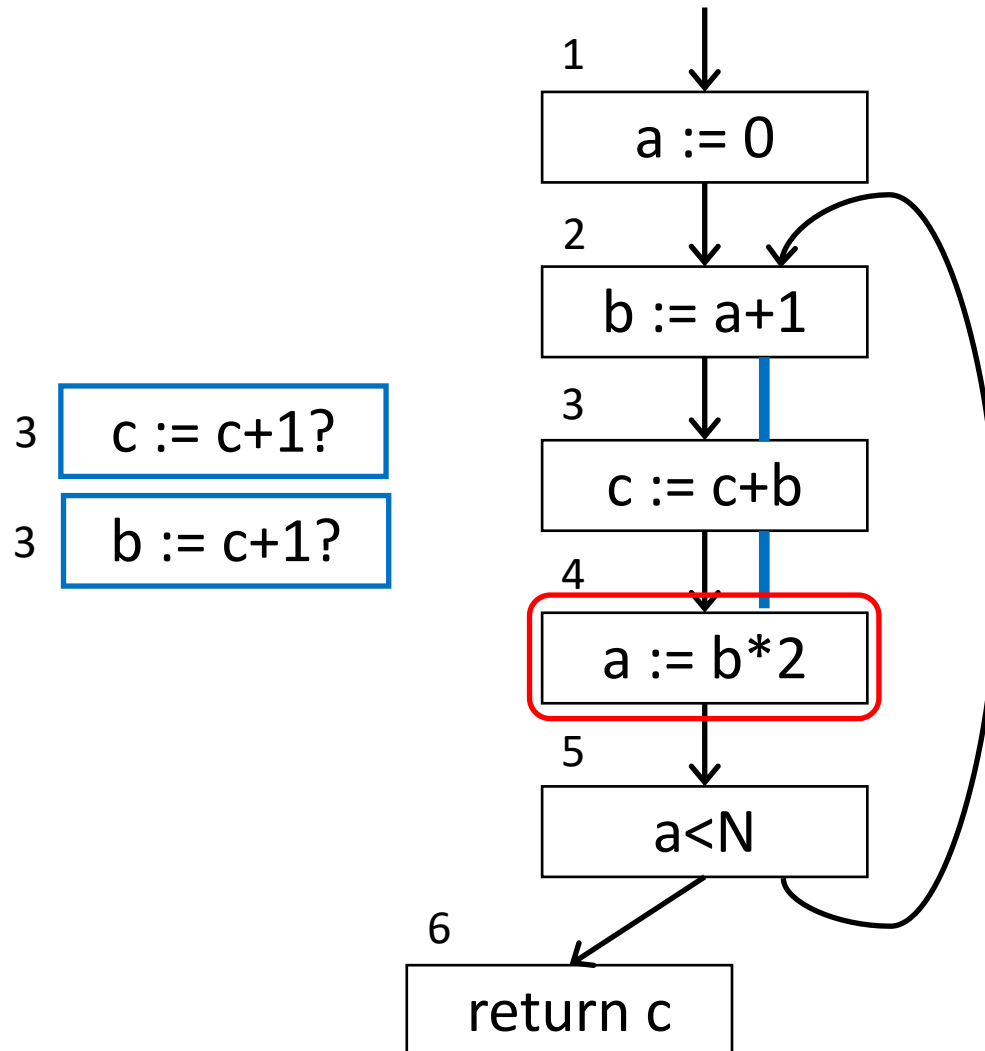
$$\text{return } c$$

```



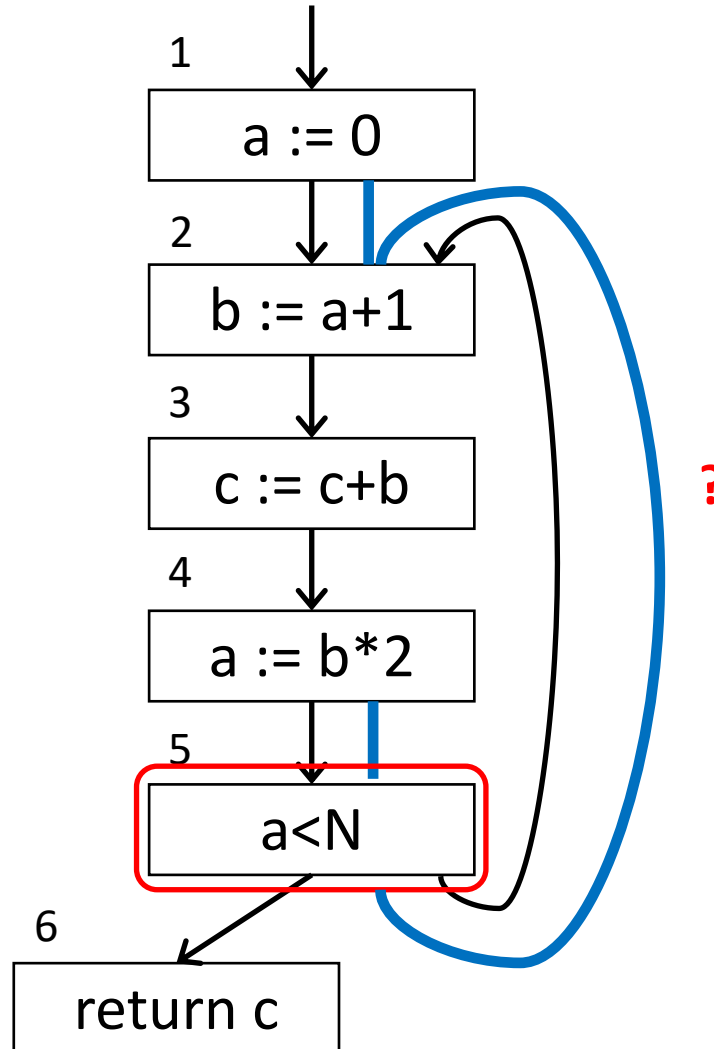
# Liveness Analysis - Example

- The liveness of variable **b**



# Liveness Analysis - Example

- The liveness of variable **a**

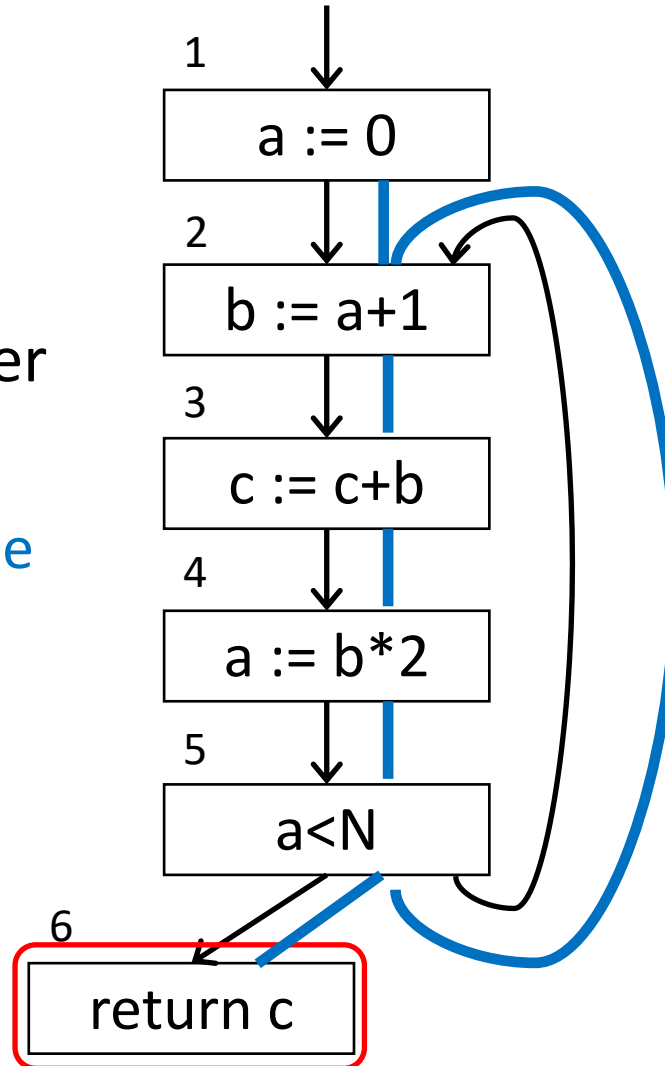


# Liveness Analysis - Example

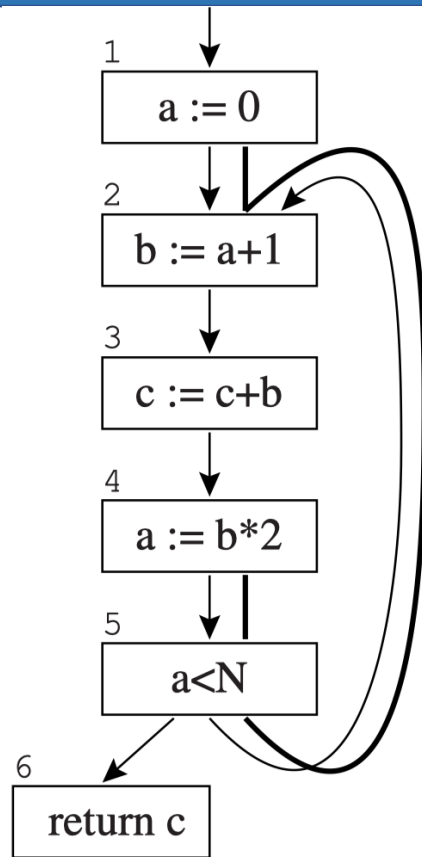
- The liveness of variable **c**

Variable **c** may be:

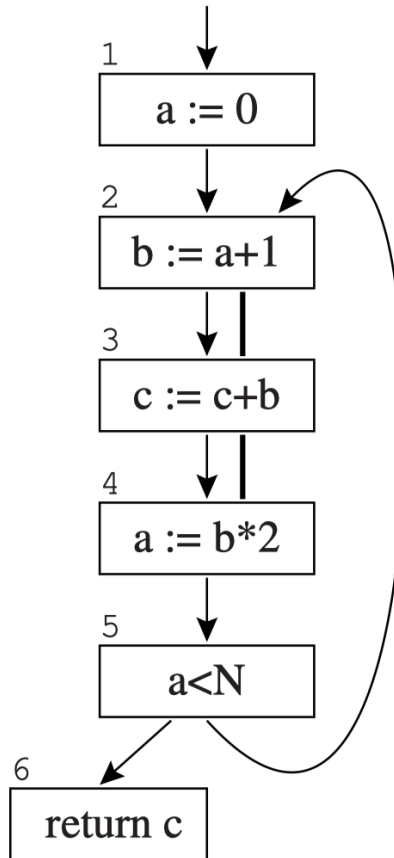
- a function parameter
- a local variable
  - undefined variable



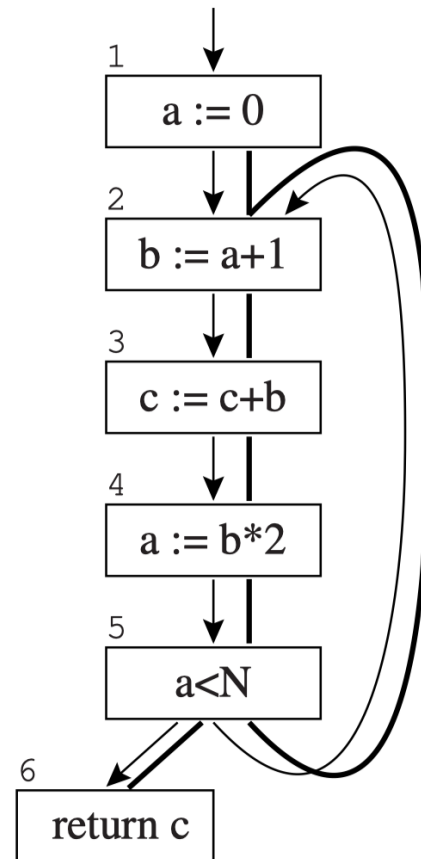
# Liveness Analysis - Example



(a)



(b)

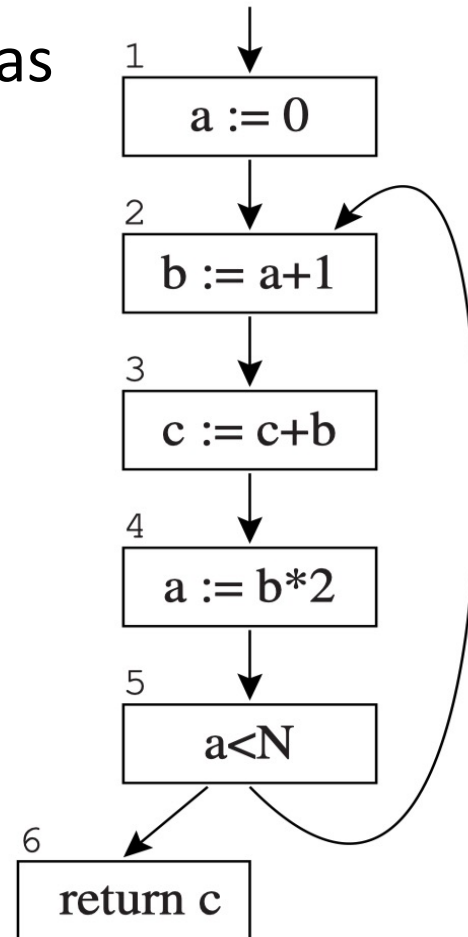


(c)

- Since **a** and **b** are never live at the same time. They can be held in the same register.

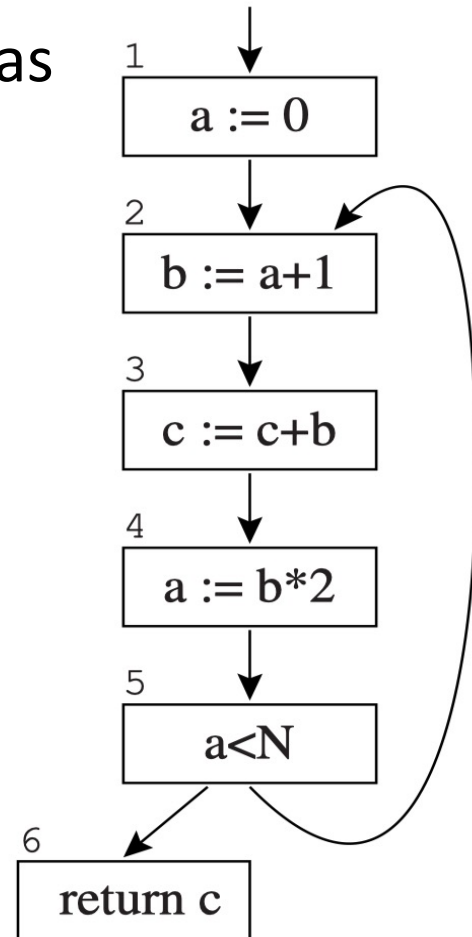
# Flow Graph Terminology

- Liveness of variables “flows” around the edges of the control-flow graph.
- Liveness analysis is an example of **dataflow** problem.
- Flow graph terminology: a flow-graph node has
  - **out-edges**: lead to successor nodes
  - **in-edges**: come from predecessor nodes
  - **pred[n]**: all the predecessors of node **n**
  - **succ[n]**: the set of successors of node **n**
- Example
  - out-edges of node 5: ?
  - $\text{succ}[5] = ?$
  - in-edges of 2 are ?
  - $\text{pred}[2] = ?$



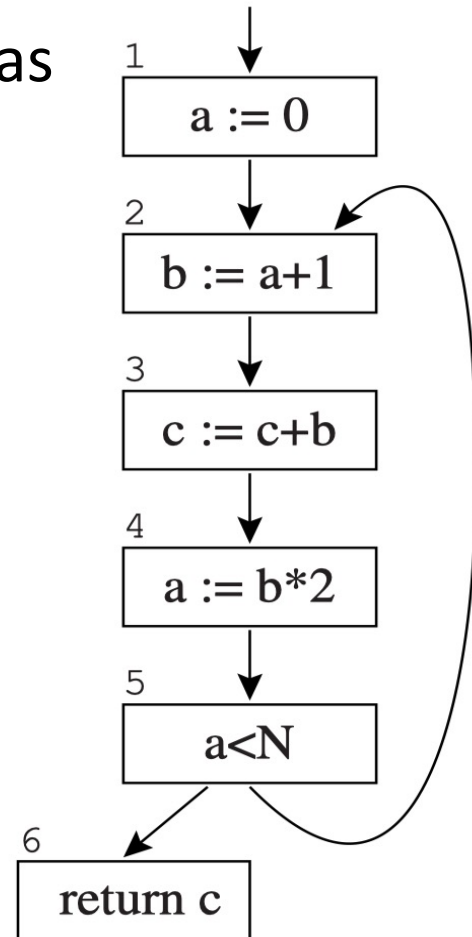
# Flow Graph Terminology

- Liveness of variables “flows” around the edges of the control-flow graph.
- Liveness analysis is an example of **dataflow** problem.
- Flow graph terminology: a flow-graph node has
  - **out-edges**: lead to successor nodes
  - **in-edges**: come from predecessor nodes
  - **pred[n]**: all the predecessors of node **n**
  - **succ[n]**: the set of successors of node **n**
- Example
  - out-edges of node 5: 5->6, 5->2
  - succ[5] = ?
  - in-edges of 2 are ?
  - pred[2] = ?



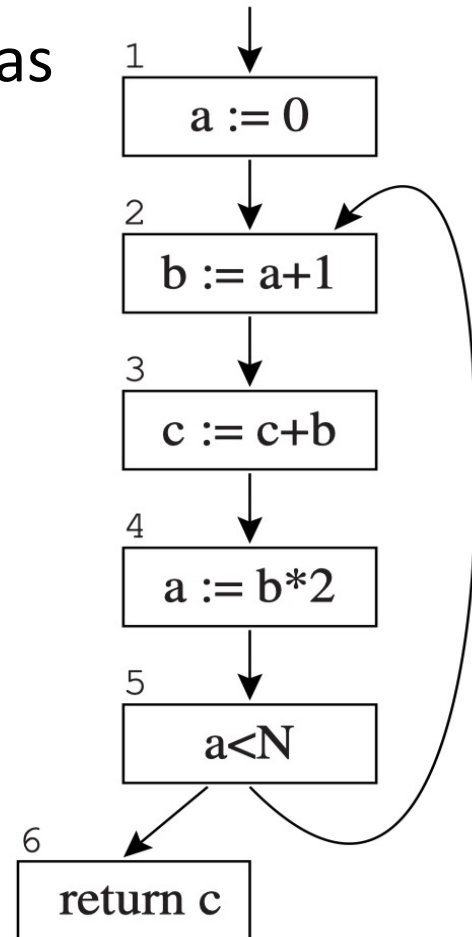
# Flow Graph Terminology

- Liveness of variables “flows” around the edges of the control-flow graph.
- Liveness analysis is an example of **dataflow** problem.
- Flow graph terminology: a flow-graph node has
  - **out-edges**: lead to successor nodes
  - **in-edges**: come from predecessor nodes
  - **pred[n]**: all the predecessors of node **n**
  - **succ[n]**: the set of successors of node **n**
- Example
  - out-edges of node 5: 5->6, 5->2
  - $\text{succ}[5] = \{2, 6\}$
  - in-edges of 2 are ?
  - $\text{pred}[2] = ?$



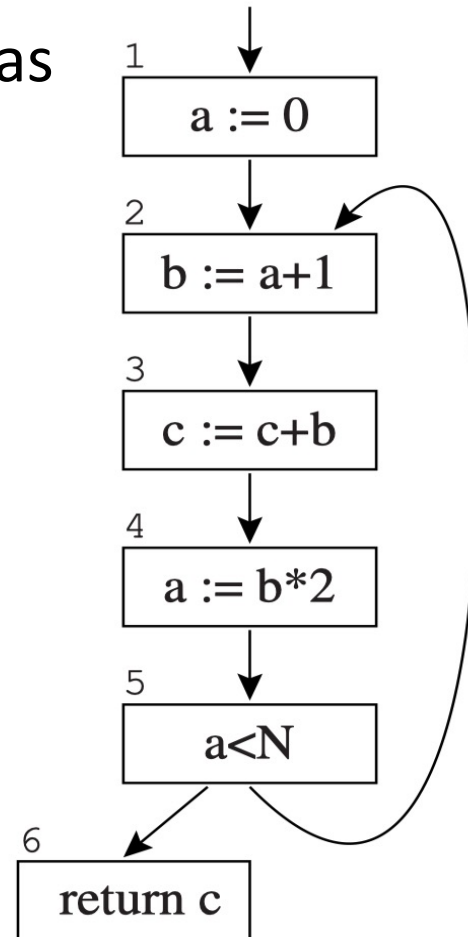
# Flow Graph Terminology

- Liveness of variables “flows” around the edges of the control-flow graph.
- Liveness analysis is an example of **dataflow** problem.
- Flow graph terminology: a flow-graph node has
  - **out-edges**: lead to successor nodes
  - **in-edges**: come from predecessor nodes
  - **pred[n]**: all the predecessors of node **n**
  - **succ[n]**: the set of successors of node **n**
- Example
  - out-edges of node 5: 5->6, 5->2
  - $\text{succ}[5] = \{2, 6\}$
  - in-edges of 2 are 5->2, 1->2
  - $\text{pred}[2] = ?$



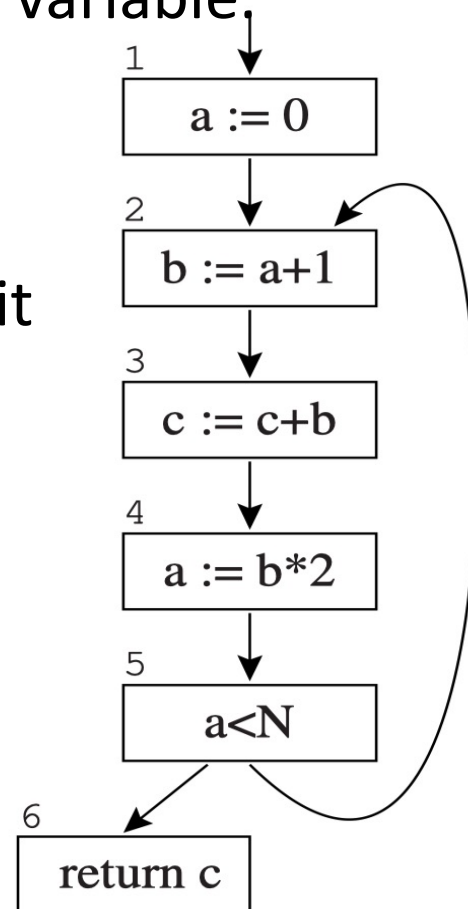
# Flow Graph Terminology

- Liveness of variables “flows” around the edges of the control-flow graph.
- Liveness analysis is an example of **dataflow** problem.
- Flow graph terminology: a flow-graph node has
  - **out-edges**: lead to successor nodes
  - **in-edges**: come from predecessor nodes
  - **pred[n]**: all the predecessors of node **n**
  - **succ[n]**: the set of successors of node **n**
- Example
  - out-edges of node 5: 5->6, 5->2
  - $\text{succ}[5] = \{2, 6\}$
  - in-edges of 2 are 5->2, 1->2
  - $\text{pred}[2] = \{1, 5\}$



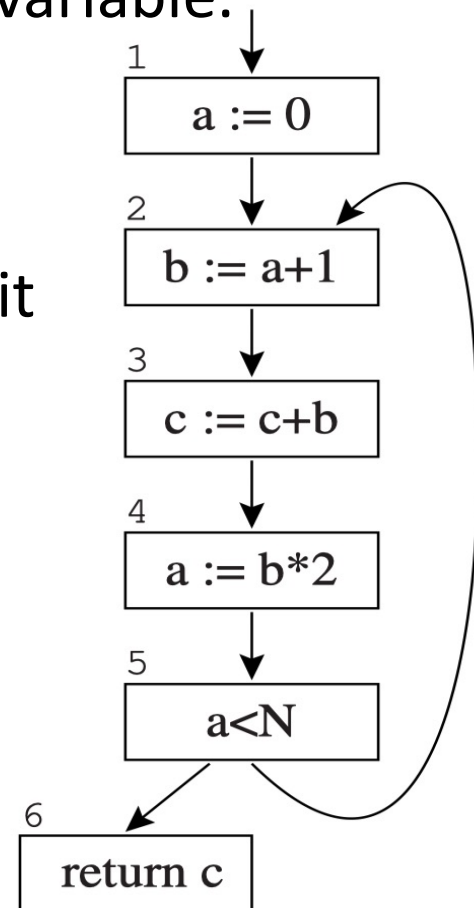
# Uses and Defs

- An assignment to a variable or temporary **defines** that variable.
- An occurrence of a variable on the right-hand side of an assignment (or in other expressions) **uses** the variable.
- **def** of a *variable*: the set of graph nodes that define it
- **def** of a *graph node*: the set of variables that it defines
- **use** of a *variable* or *graph node* is similar.
- Example
  - $\text{def}(3) = ?$ ,  $\text{def}(a) = ?$
  - $\text{use}(3) = ?$ ,  $\text{use}(a) = ?$



# Uses and Defs

- An assignment to a variable or temporary **defines** that variable.
- An occurrence of a variable on the right-hand side of an assignment (or in other expressions) **uses** an variable.
- **def** of a *variable*: the set of graph nodes that define it
- **def** of a *graph node*: the set of variables that it defines
- **use** of a *variable* or *graph node* is similar.
- Example
  - $\text{def}(3) = \{c\}$ ,  $\text{def}(a) = ?$
  - $\text{use}(3) = ?$ ,  $\text{use}(a) = ?$



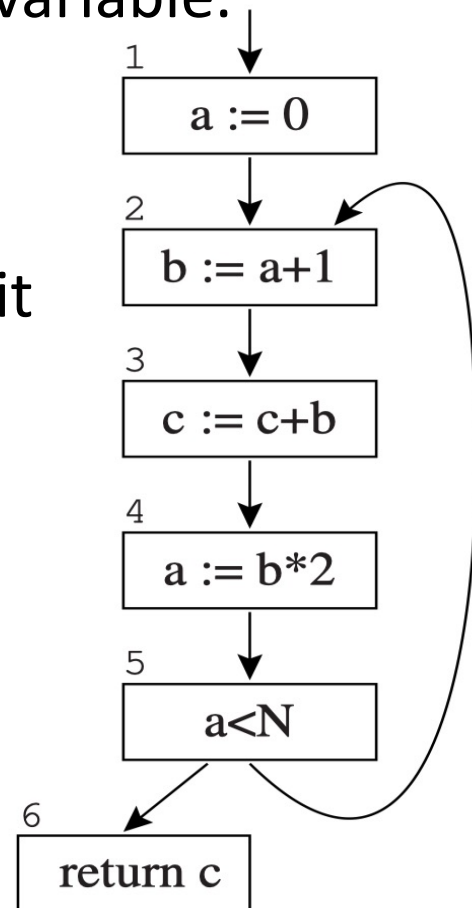
# Uses and Defs

- An assignment to a variable or temporary **defines** that variable.
- An occurrence of a variable on the right-hand side of an assignment (or in other expressions) **uses** an variable.

- **def** of a *variable*: the set of graph nodes that define it
- **def** of a *graph node*: the set of variables that it defines
- **use** of a *variable* or *graph node* is similar.

- Example

- $\text{def}(3) = \{c\}$ ,  $\text{def}(a) = \{1, 4\}$
- $\text{use}(3) = ?$ ,  $\text{use}(a) = ?$



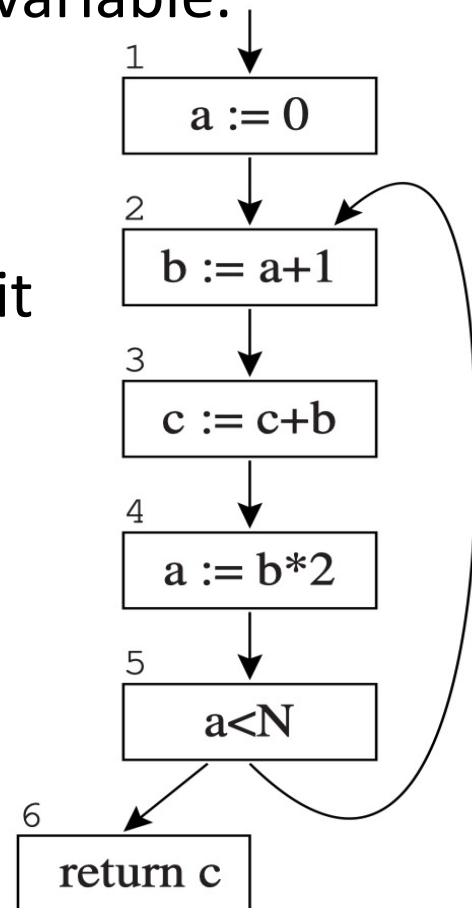
# Uses and Defs

- An assignment to a variable or temporary **defines** that variable.
- An occurrence of a variable on the right-hand side of an assignment (or in other expressions) **uses** an variable.

- **def** of a *variable*: the set of graph nodes that define it
- **def** of a *graph node*: the set of variables that it defines
- **use** of a *variable* or *graph node* is similar.

- Example

- $\text{def}(3) = \{c\}$ ,  $\text{def}(a) = \{1, 4\}$
- $\text{use}(3) = \{b, c\}$ ,  $\text{use}(a) = ?$



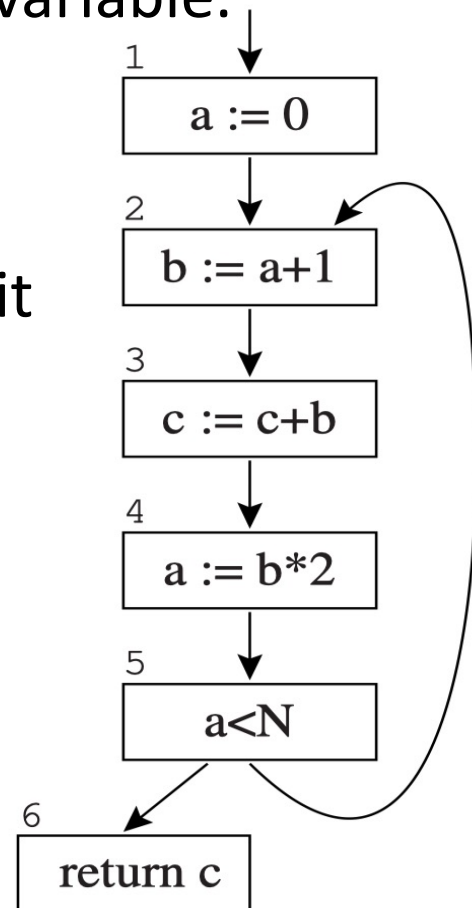
# Uses and Defs

- An assignment to a variable or temporary **defines** that variable.
- An occurrence of a variable on the right-hand side of an assignment (or in other expressions) **uses** an variable.

- **def** of a *variable*: the set of graph nodes that define it
- **def** of a *graph node*: the set of variables that it defines
- **use** of a *variable* or *graph node* is similar.

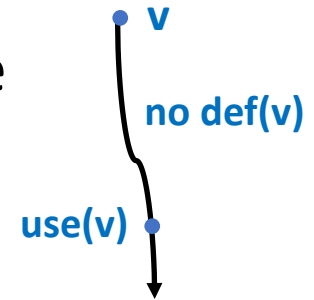
- Example

- $\text{def}(3) = \{c\}$ ,  $\text{def}(a) = \{1, 4\}$
- $\text{use}(3) = \{b, c\}$ ,  $\text{use}(a) = \{2, 5\}$



# Liveness

- **Liveness**: A variable is **live** on an edge if there is a **directed path** from that **edge** to a **use** of the variable that **does not go through any def**.
  - this variable will be used later
  - this variable will not be re-defined before being used
- **Live-in**: A variable is **live-in** at a node if it is live on **any** of the **in-edges** of that node;
- **Live-out**: A variable is **live-out** at a node if it is live on **any** of the **out-edges** of the node.
- Notions:
  - **in[n]**: the live-in set of node **n** (the variables that is live-in at node **n**)
  - **out[n]**: the live-out set of node **n** (the variables that is live-out at node **n**)



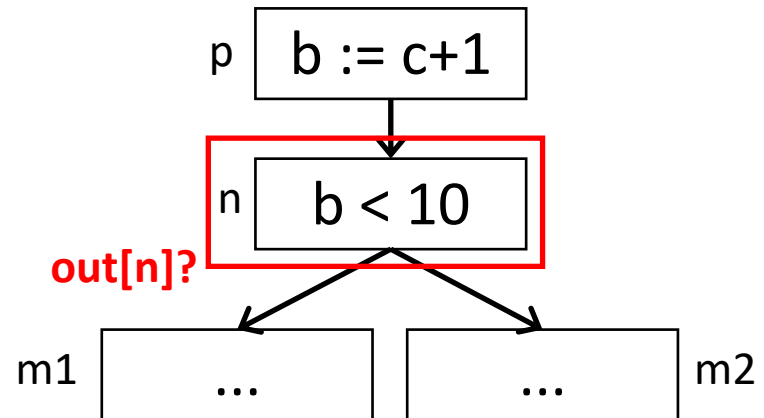
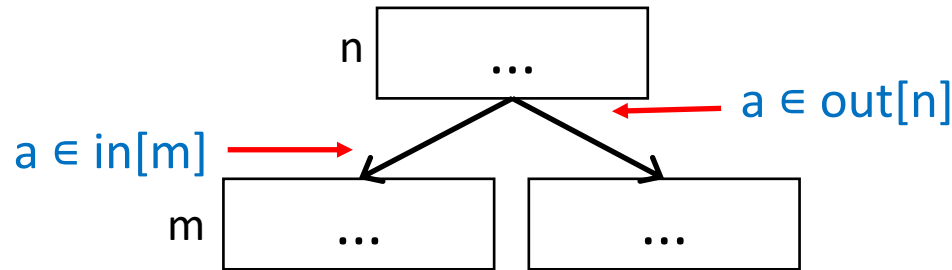
# Outline

- Liveness Analysis
  - Why, How, and What
  - **Dataflow Equation for Liveness**
  - Solving the Equations
- More Discussions
- Interference Graphs
- Liveness in the Tiger Compiler

# Calculation of Liveness

Liveness information ( $\text{in}[n]$  and  $\text{out}[n]$ ) can be calculated from  $\text{use}$  and  $\text{def}$ :

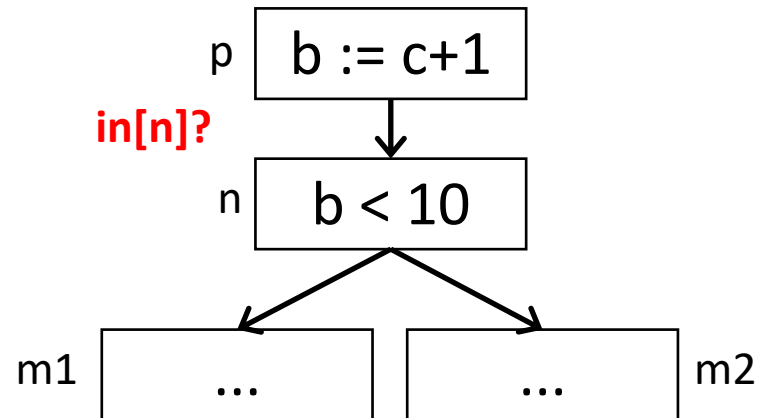
- **Rule 1:** If  $a \in \text{in}[m]$  for  $\forall m \in \text{succ}[n]$ , then  $a \in \text{out}[n]$



# Calculation of Liveness

Liveness information ( $\text{in}[n]$  and  $\text{out}[n]$ ) can be calculated from  $\text{use}$  and  $\text{def}$ :

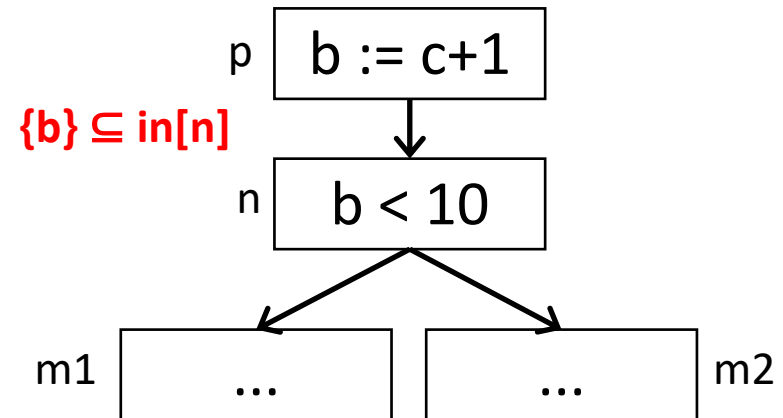
- **Rule 1:** If  $a \in \text{in}[m]$  for  $\forall m \in \text{succ}[n]$ , then  $a \in \text{out}[n]$
- **Rule 2:** If  $a \in \text{use}[n]$ , then  $a \in \text{in}[n]$ 
  - i.e., if statement  $n$  uses variable  $a$ ,  $a$  is live on entry of  $n$



# Calculation of Liveness

Liveness information ( $\text{in}[n]$  and  $\text{out}[n]$ ) can be calculated from  $\text{use}$  and  $\text{def}$ :

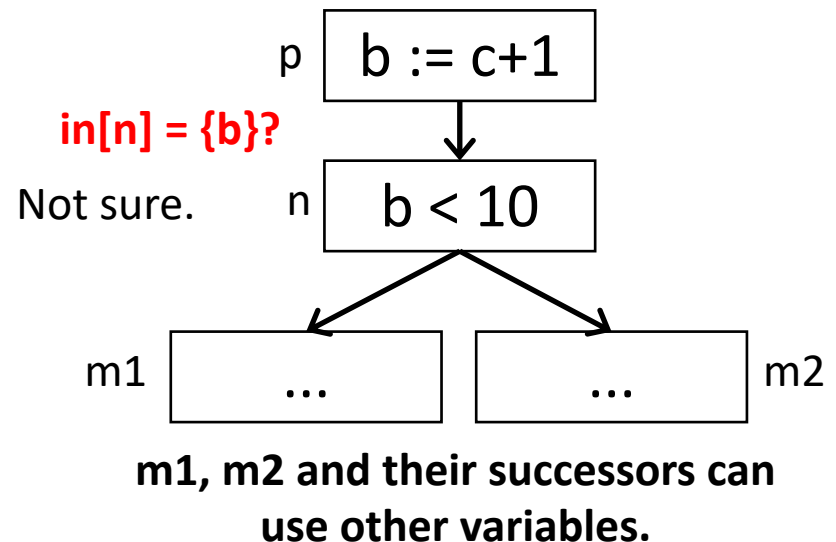
- **Rule 1:** If  $a \in \text{in}[m]$  for  $\forall m \in \text{succ}[n]$ , then  $a \in \text{out}[n]$
- **Rule 2:** If  $a \in \text{use}[n]$ , then  $a \in \text{in}[n]$ 
  - i.e., if statement  $n$  uses variable  $a$ ,  $a$  is live on entry of  $n$



# Calculation of Liveness

Liveness information ( $\text{in}[n]$  and  $\text{out}[n]$ ) can be calculated from  $\text{use}$  and  $\text{def}$ :

- **Rule 1:** If  $a \in \text{in}[m]$  for  $\forall m \in \text{succ}[n]$ , then  $a \in \text{out}[n]$
- **Rule 2:** If  $a \in \text{use}[n]$ , then  $a \in \text{in}[n]$ 
  - i.e., if statement  $n$  uses variable  $a$ ,  $a$  is live on entry of  $n$
- **Rule 3:** If  $a \in \text{out}[n]$ , then  $a \in \text{in}[n]$  ?
  - $b \in \text{out}[p]$ . Is  $b \in \text{in}[p]$ ?

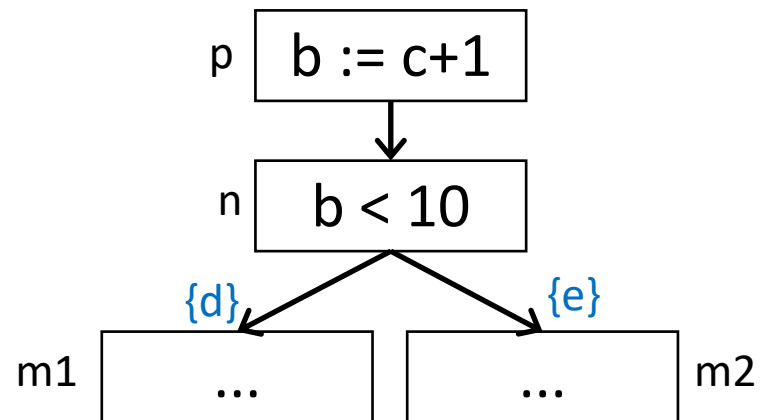


# Calculation of Liveness

Liveness information ( $\text{in}[n]$  and  $\text{out}[n]$ ) can be calculated from  $\text{use}$  and  $\text{def}$ :

- **Rule 1:** If  $a \in \text{in}[m]$  for  $\forall m \in \text{succ}[n]$ , then  $a \in \text{out}[n]$
- **Rule 2:** If  $a \in \text{use}[n]$ , then  $a \in \text{in}[n]$ 
  - i.e., if statement  $n$  uses variable  $a$ ,  $a$  is live on entry of  $n$
- **Rule 3:** If  $a \in \text{out}[n]$  and  $a \notin \text{def}[n]$ , then  $a \in \text{in}[n]$
- **Example:**

- Suppose:
  - $\text{in}[m1] = \{d\}$ ,  $\text{in}[m2] = \{e\}$
- $\text{out}[n] = ?$
- $\text{in}[n] = ?$
- $\text{out}[p] = ?$
- $\text{in}[p] = ?$

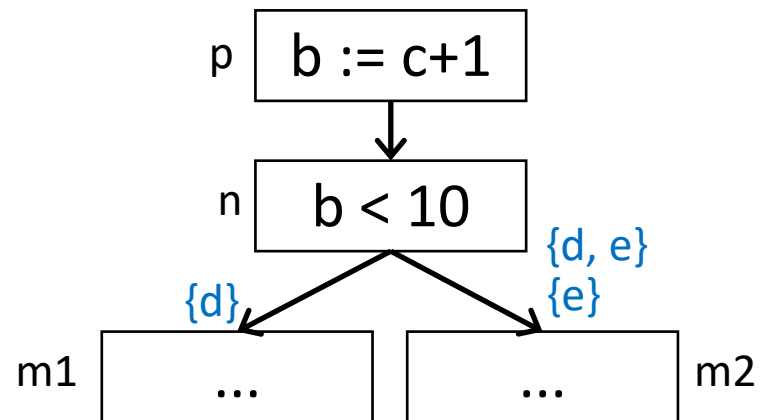


# Calculation of Liveness

Liveness information ( $\text{in}[n]$  and  $\text{out}[n]$ ) can be calculated from  $\text{use}$  and  $\text{def}$ :

- **Rule 1:** If  $a \in \text{in}[m]$  for  $\forall m \in \text{succ}[n]$ , then  $a \in \text{out}[n]$
- **Rule 2:** If  $a \in \text{use}[n]$ , then  $a \in \text{in}[n]$ 
  - i.e., if statement  $n$  uses variable  $a$ ,  $a$  is live on entry of  $n$
- **Rule 3:** If  $a \in \text{out}[n]$  and  $a \notin \text{def}[n]$ , then  $a \in \text{in}[n]$
- **Example:**

- Suppose:
  - $\text{in}[m1] = \{d\}$ ,  $\text{in}[m2] = \{e\}$
- $\text{out}[n] = \{d, e\}$
- $\text{in}[n] = ?$
- $\text{out}[p] = ?$
- $\text{in}[p] = ?$

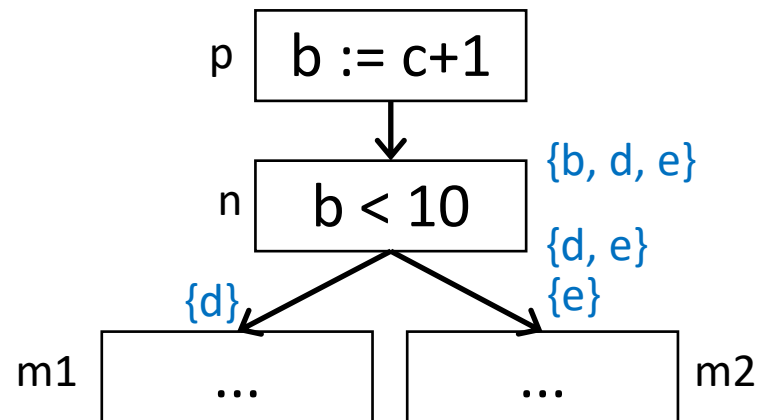


# Calculation of Liveness

Liveness information ( $\text{in}[n]$  and  $\text{out}[n]$ ) can be calculated from  $\text{use}$  and  $\text{def}$ :

- **Rule 1:** If  $a \in \text{in}[m]$  for  $\forall m \in \text{succ}[n]$ , then  $a \in \text{out}[n]$
- **Rule 2:** If  $a \in \text{use}[n]$ , then  $a \in \text{in}[n]$ 
  - i.e., if statement  $n$  uses variable  $a$ ,  $a$  is live on entry of  $n$
- **Rule 3:** If  $a \in \text{out}[n]$  and  $a \notin \text{def}[n]$ , then  $a \in \text{in}[n]$
- **Example:**

- Suppose:
  - $\text{in}[m1] = \{d\}$ ,  $\text{in}[m2] = \{e\}$
- $\text{out}[n] = \{d, e\}$
- $\text{in}[n] = \{b\} \cup (\{d, e\} - \emptyset) = \{b, d, e\}$
- $\text{out}[p] = ?$
- $\text{in}[p] = ?$

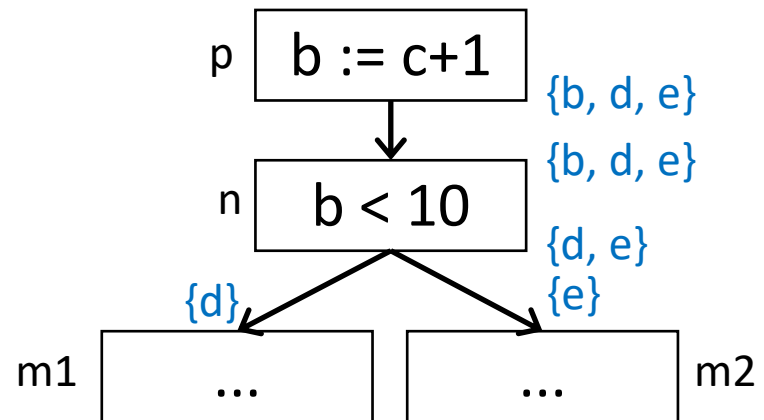


# Calculation of Liveness

Liveness information ( $\text{in}[n]$  and  $\text{out}[n]$ ) can be calculated from  $\text{use}$  and  $\text{def}$ :

- **Rule 1:** If  $a \in \text{in}[m]$  for  $\forall m \in \text{succ}[n]$ , then  $a \in \text{out}[n]$
- **Rule 2:** If  $a \in \text{use}[n]$ , then  $a \in \text{in}[n]$ 
  - i.e., if statement  $n$  uses variable  $a$ ,  $a$  is live on entry of  $n$
- **Rule 3:** If  $a \in \text{out}[n]$  and  $a \notin \text{def}[n]$ , then  $a \in \text{in}[n]$
- **Example:**

- Suppose:
  - $\text{in}[m1] = \{d\}$ ,  $\text{in}[m2] = \{e\}$
- $\text{out}[n] = \{d, e\}$
- $\text{in}[n] = \{b, d, e\}$
- $\text{out}[p] = \{b, d, e\}$
- $\text{in}[p] = ?$

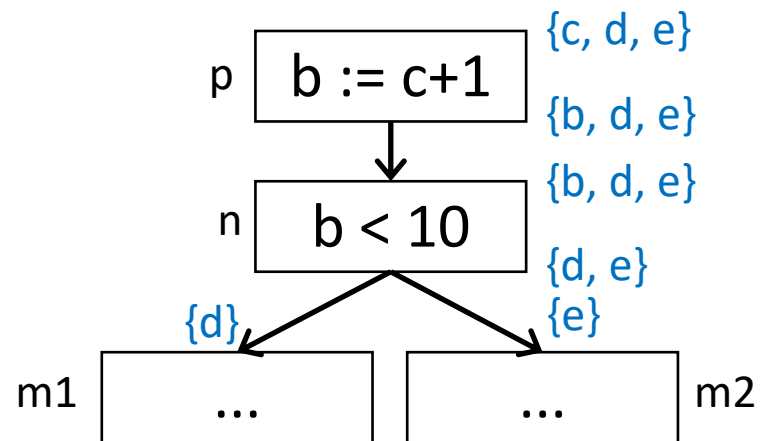


# Calculation of Liveness

Liveness information ( $\text{in}[n]$  and  $\text{out}[n]$ ) can be calculated from  $\text{use}$  and  $\text{def}$ :

- **Rule 1:** If  $a \in \text{in}[m]$  for  $\forall m \in \text{succ}[n]$ , then  $a \in \text{out}[n]$
- **Rule 2:** If  $a \in \text{use}[n]$ , then  $a \in \text{in}[n]$ 
  - i.e., if statement  $n$  uses variable  $a$ ,  $a$  is live on entry of  $n$
- **Rule 3:** If  $a \in \text{out}[n]$  and  $a \notin \text{def}[n]$ , then  $a \in \text{in}[n]$
- **Example:**

- Suppose:
  - $\text{in}[m1] = \{d\}$ ,  $\text{in}[m2] = \{e\}$
- $\text{out}[n] = \{d, e\}$
- $\text{in}[n] = \{b, d, e\}$
- $\text{out}[p] = \{b, d, e\}$
- $\text{in}[p] = \{c\} \cup (\{b, d, e\} - \{b\}) = \{c, d, e\}$



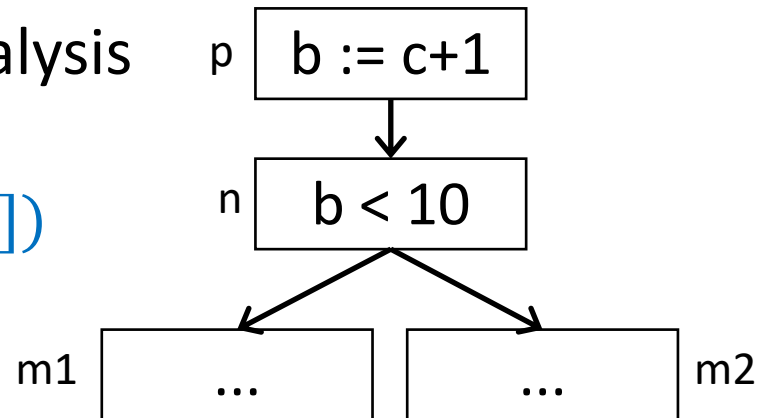
# Calculation of Liveness

Liveness information ( $\text{in}[n]$  and  $\text{out}[n]$ ) can be calculated from  $\text{use}$  and  $\text{def}$ :

- **Rule 1:** If  $a \in \text{in}[m]$  for  $\forall m \in \text{succ}[n]$ , then  $a \in \text{out}[n]$
- **Rule 2:** If  $a \in \text{use}[n]$ , then  $a \in \text{in}[n]$ 
  - i.e., if statement  $n$  uses variable  $a$ ,  $a$  is live on entry of  $n$
- **Rule 3:** If  $a \in \text{out}[n]$  and  $a \notin \text{def}[n]$ , then  $a \in \text{in}[n]$

- Dataflow equations for liveness analysis

$$\begin{aligned}\text{in}[n] &= \text{use}[n] \cup (\text{out}[n] - \text{def}[n]) \\ \text{out}[n] &= \bigcup_{s \in \text{succ}[n]} \text{in}[s]\end{aligned}$$



# Outline

- Liveness Analysis
  - Why, How, and What
  - Dataflow Equation for Liveness
  - **Solving the Equations**
- More Discussions
- Interference Graph
- Liveness in the Tiger Compiler

# Calculation of Liveness

$$\begin{aligned} \text{in}[n] &= \text{use}[n] \cup (\text{out}[n] - \text{def}[n]) \\ \text{out}[n] &= \bigcup_{s \in \text{succ}[n]} \text{in}[s] \end{aligned}$$

The algorithm for finding a solution to these equations by iteration:

```
for each n
  in[n] ← {}; out[n] ← {}
repeat
  for each n
    in'[n] ← in[n]; out'[n] ← out[n]
    in[n] ← use[n] ∪ (out[n] - def[n])
    out[n] ←  $\bigcup_{s \in \text{succ}[n]} \text{in}[s]$ 
  until in'[n] = in[n] and out'[n] = out[n] for all n
```

- $\text{use}[n]$  and  $\text{def}[n]$  can be obtained by analyzing statement  $n$ , and thus are treated as known.

# Solving Dataflow Equations

**for each**  $n$

$\text{in}[n] \leftarrow \{\}; \text{out}[n] \leftarrow \{\}$

**repeat**

**for each**  $n$

$\text{in}'[n] \leftarrow \text{in}[n]; \text{out}'[n] \leftarrow \text{out}[n]$

$\text{in}[n] \leftarrow \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$

$\text{out}[n] \leftarrow \bigcup_{s \in \text{succ}[n]} \text{in}[s]$

**until**  $\text{in}'[n] = \text{in}[n]$  and  $\text{out}'[n] = \text{out}[n]$  for all  $n$

- Start with a “rough” approximation to the answer

# Solving Dataflow Equations

```
for each n
  in[n]  $\leftarrow$  {}; out[n]  $\leftarrow$  {}
repeat
  for each n
    in'[n]  $\leftarrow$  in[n]; out'[n]  $\leftarrow$  out[n]
    in[n]  $\leftarrow$  use[n]  $\cup$  (out[n] - def[n])
    out[n]  $\leftarrow$   $\bigcup_{s \in \text{succ}[n]} \text{in}[s]$ 
  until in'[n] = in[n] and out'[n] = out[n] for all n
```

- Start with a “rough” approximation to the answer
- Iteratively re-compute in[n] and out[n]
  - Each iteration will add variables to in[n] and out[n]
  - i.e., the live variable sets will increase monotonically.
- Keep going until a fixed point has been reached

# Calculation of Liveness

- Example: following **forward** control-flow edges

|   |     |     | 1 <sup>st</sup> | 2 <sup>nd</sup> | 3 <sup>rd</sup> | 4 <sup>th</sup> | 5 <sup>th</sup> | 6 <sup>th</sup> | 7 <sup>th</sup> |     |
|---|-----|-----|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----|
|   | use | def | in              | out             | in              | out             | in              | out             | in              | out |
| 1 |     | a   |                 |                 |                 |                 |                 |                 |                 |     |
| 2 | a   | b   |                 |                 |                 |                 |                 |                 |                 |     |
| 3 | bc  | c   |                 |                 |                 |                 |                 |                 |                 |     |
| 4 | b   | a   |                 |                 |                 |                 |                 |                 |                 |     |
| 5 | a   |     |                 |                 |                 |                 |                 |                 |                 |     |
| 6 | c   |     |                 |                 |                 |                 |                 |                 |                 |     |

1

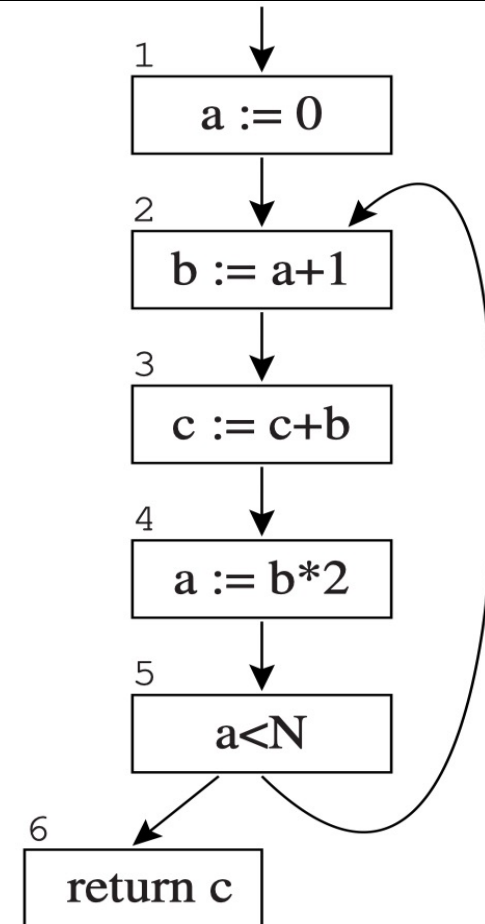
a := 0

2

b := a+1

3

c := c+b



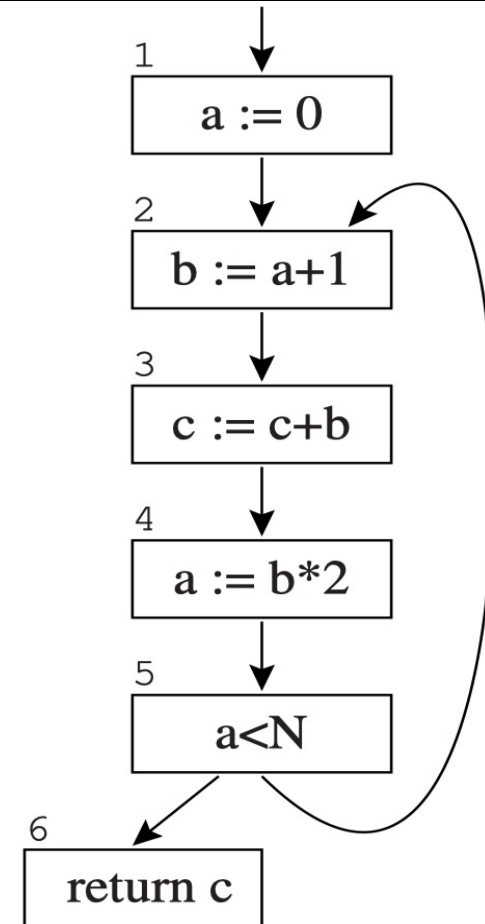
$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$

# Calculation of Liveness

- Example: following **forward** control-flow edges

|   | use | def | 1 <sup>st</sup> |     | 2 <sup>nd</sup> |     | 3 <sup>rd</sup> |     | 4 <sup>th</sup> |     | 5 <sup>th</sup> |     | 6 <sup>th</sup> |     | 7 <sup>th</sup> |     |
|---|-----|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|
|   |     |     | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out |
| 1 |     | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 2 | a   | b   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 3 | bc  | c   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 4 | b   | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 5 | a   |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 6 | c   |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |



$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$

# Calculation of Liveness

- Example: following **forward** control-flow edges

|   |     |     | 1 <sup>st</sup> | 2 <sup>nd</sup> | 3 <sup>rd</sup> | 4 <sup>th</sup> | 5 <sup>th</sup> | 6 <sup>th</sup> | 7 <sup>th</sup> |     |
|---|-----|-----|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----|
|   | use | def | in              | out             | in              | out             | in              | out             | in              | out |
| 1 |     | a   |                 |                 |                 |                 |                 |                 |                 |     |
| 2 | a   | b   | a               |                 |                 |                 |                 |                 |                 |     |
| 3 | bc  | c   |                 |                 |                 |                 |                 |                 |                 |     |
| 4 | b   | a   |                 |                 |                 |                 |                 |                 |                 |     |
| 5 | a   |     |                 |                 |                 |                 |                 |                 |                 |     |
| 6 | c   |     |                 |                 |                 |                 |                 |                 |                 |     |

1

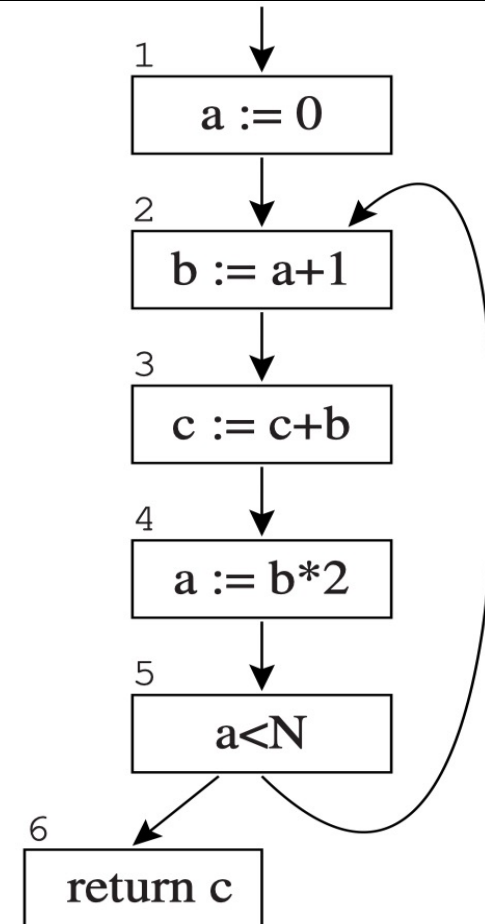
a := 0

2

b := a+1

3

c := c+b



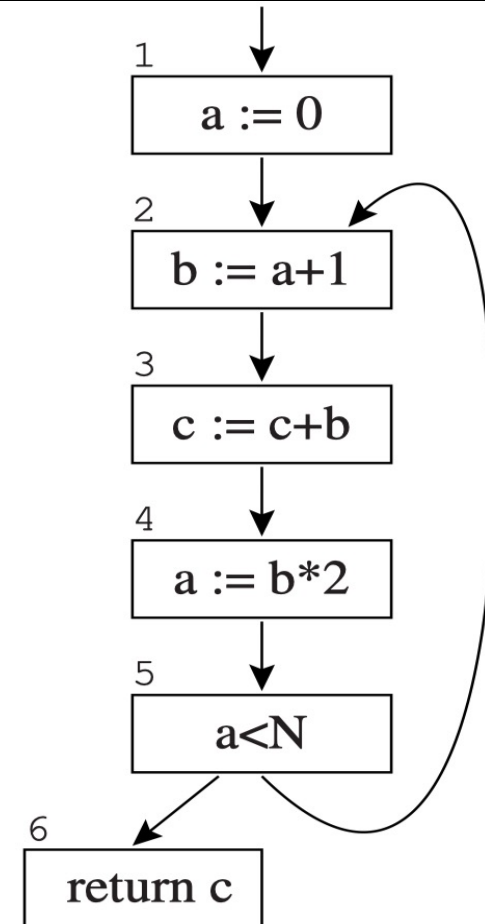
$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$

# Calculation of Liveness

- Example: following **forward** control-flow edges

|   |     |     | 1 <sup>st</sup> |     | 2 <sup>nd</sup> |     | 3 <sup>rd</sup> |     | 4 <sup>th</sup> |     | 5 <sup>th</sup> |     | 6 <sup>th</sup> |     | 7 <sup>th</sup> |     |
|---|-----|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|
|   | use | def | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out |
| 1 |     | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 2 | a   | b   | a               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 3 | bc  | c   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 4 | b   | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 5 | a   |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 6 | c   |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |



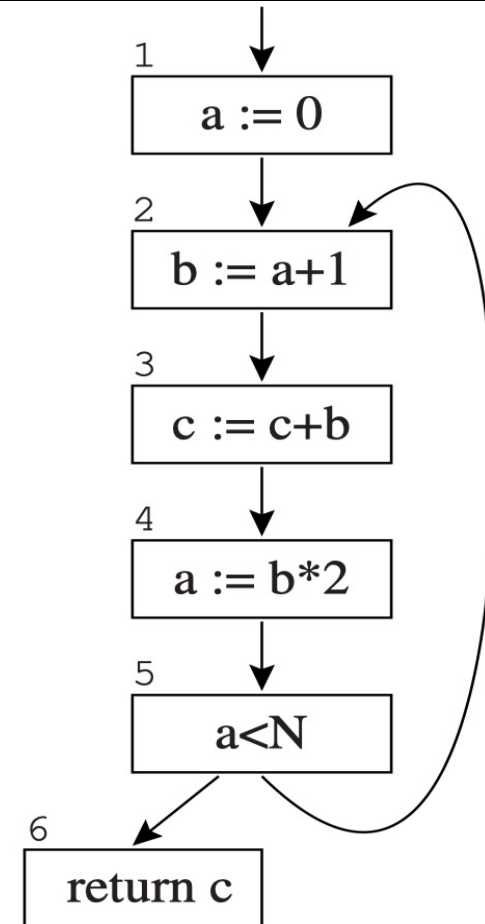
$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$

# Calculation of Liveness

- Example: following **forward** control-flow edges

|   |     |     | 1 <sup>st</sup> |     | 2 <sup>nd</sup> |     | 3 <sup>rd</sup> |     | 4 <sup>th</sup> |     | 5 <sup>th</sup> |     | 6 <sup>th</sup> |     | 7 <sup>th</sup> |     |
|---|-----|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|
|   | use | def | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out |
| 1 |     | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 2 | a   | b   | a               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 3 | bc  | c   | bc              |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 4 | b   | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 5 | a   |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 6 | c   |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |



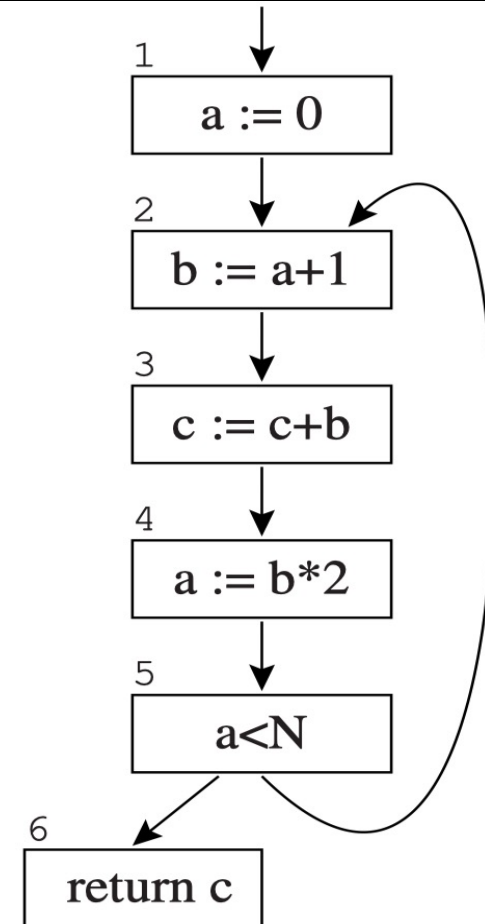
$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$

# Calculation of Liveness

- Example: following **forward** control-flow edges

|   | use | def | 1 <sup>st</sup> |     | 2 <sup>nd</sup> |     | 3 <sup>rd</sup> |     | 4 <sup>th</sup> |     | 5 <sup>th</sup> |     | 6 <sup>th</sup> |     | 7 <sup>th</sup> |     |
|---|-----|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|
|   |     |     | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out |
| 1 |     | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 2 | a   | b   | a               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 3 | bc  | c   | bc              |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 4 | b   | a   | b               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 5 | a   |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 6 | c   |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |



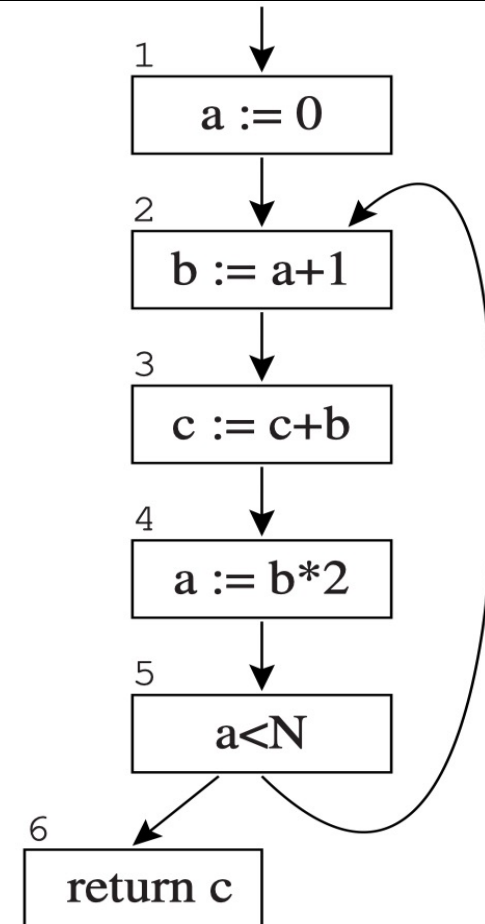
$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$

# Calculation of Liveness

- Example: following **forward** control-flow edges

|   | use | def | 1 <sup>st</sup> |     | 2 <sup>nd</sup> |     | 3 <sup>rd</sup> |     | 4 <sup>th</sup> |     | 5 <sup>th</sup> |     | 6 <sup>th</sup> |     | 7 <sup>th</sup> |     |
|---|-----|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|
|   |     |     | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out |
| 1 |     | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 2 | a   | b   | a               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 3 | bc  | c   | bc              |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 4 | b   | a   | b               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 5 | a   |     | a               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 6 | c   |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |



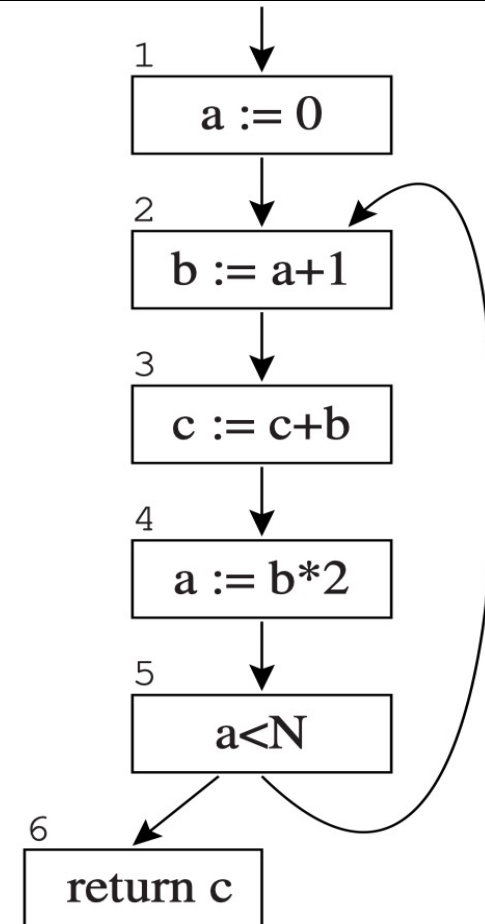
$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$

# Calculation of Liveness

- Example: following **forward** control-flow edges

|   | use | def | 1 <sup>st</sup> |     | 2 <sup>nd</sup> |     | 3 <sup>rd</sup> |     | 4 <sup>th</sup> |     | 5 <sup>th</sup> |     | 6 <sup>th</sup> |     | 7 <sup>th</sup> |     |
|---|-----|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|
|   |     |     | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out |
| 1 |     | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 2 | a   | b   | a               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 3 | bc  | c   | bc              |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 4 | b   | a   | b               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 5 | a   |     | a               | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 6 | c   |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |



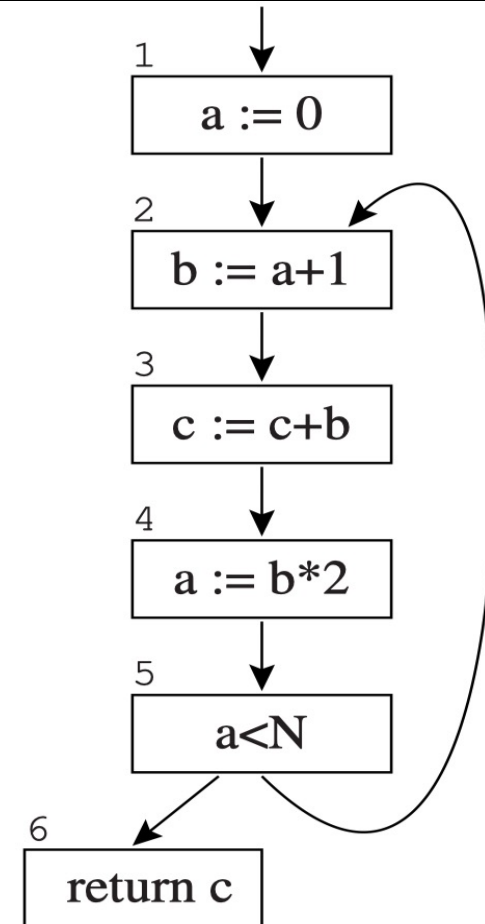
$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$

# Calculation of Liveness

- Example: following **forward** control-flow edges

|   | use | def | 1 <sup>st</sup> |     | 2 <sup>nd</sup> |     | 3 <sup>rd</sup> |     | 4 <sup>th</sup> |     | 5 <sup>th</sup> |     | 6 <sup>th</sup> |     | 7 <sup>th</sup> |     |
|---|-----|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|
|   |     |     | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out |
| 1 |     | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 2 | a   | b   | a               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 3 | bc  | c   | bc              |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 4 | b   | a   | b               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 5 | a   |     | a               | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 6 | c   |     | c               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |



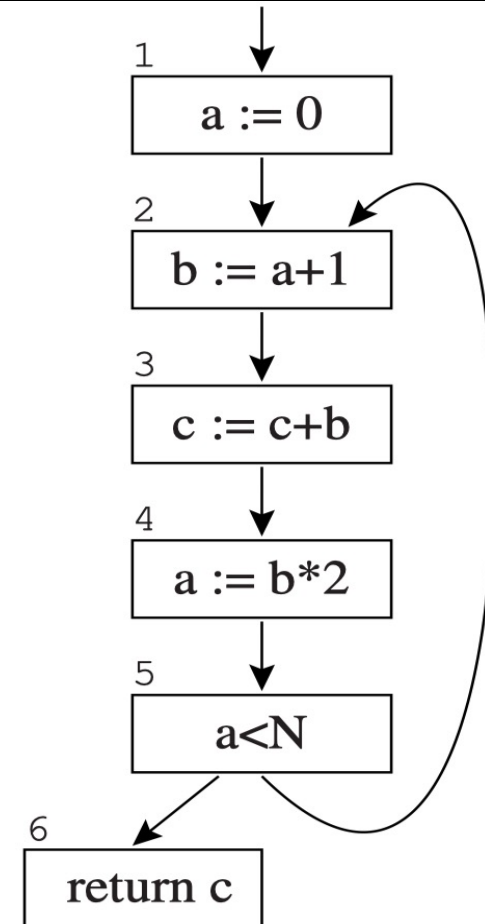
$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$

# Calculation of Liveness

- Example: following **forward** control-flow edges

|   |     |     | 1 <sup>st</sup> |     | 2 <sup>nd</sup> |     | 3 <sup>rd</sup> |     | 4 <sup>th</sup> |     | 5 <sup>th</sup> |     | 6 <sup>th</sup> |     | 7 <sup>th</sup> |     |
|---|-----|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|
|   | use | def | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out |
| 1 |     | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 2 | a   | b   | a               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 3 | bc  | c   | bc              |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 4 | b   | a   | b               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 5 | a   |     | a               | a   |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |
| 6 | c   |     | c               |     |                 |     |                 |     |                 |     |                 |     |                 |     |                 |     |



$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$

# Calculation of Liveness

- Example: following **forward** control-flow edges

|   |     |     | 1 <sup>st</sup> |     | 2 <sup>nd</sup> |     | 3 <sup>rd</sup> |     | 4 <sup>th</sup> |     | 5 <sup>th</sup> |     | 6 <sup>th</sup> |     | 7 <sup>th</sup> |     |
|---|-----|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|-----------------|-----|
|   | use | def | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out | in              | out |
| 1 |     | a   |                 |     |                 | a   |                 | a   |                 | ac  | c               | ac  | c               | ac  | c               | ac  |
| 2 | a   | b   | a               |     | a               | bc  | ac              | bc  | ac              | bc  | ac              | bc  | ac              | bc  | ac              | bc  |
| 3 | bc  | c   | bc              |     | bc              | b   | bc              | b   | bc              | b   | bc              | b   | bc              | bc  | bc              | bc  |
| 4 | b   | a   | b               |     | b               | a   | b               | a   | b               | ac  | bc              | ac  | bc              | ac  | bc              | ac  |
| 5 | a   |     | a               | a   | a               | ac  | ac              | ac  | ac              | ac  | ac              | ac  | ac              | ac  | ac              | ac  |
| 6 | c   |     | c               |     | c               |     | c               |     | c               |     | c               |     | c               |     | c               |     |

$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$

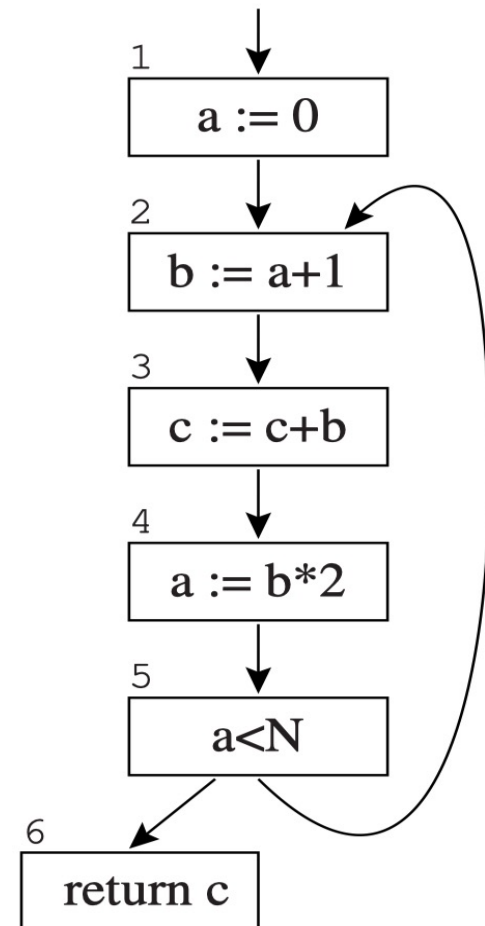
# Calculation of Liveness

- $out[n]$  is computed from  $in[s]$ .  $in[n]$  is computed from  $out[n]$
- We can speed the convergence by computing in **the opposite order** (from 6 to 1, from out to in)

|   |     |     | 1 <sup>st</sup> |    | 2 <sup>nd</sup> |    | 3 <sup>rd</sup> |    |
|---|-----|-----|-----------------|----|-----------------|----|-----------------|----|
|   | use | def | out             | in | out             | in | out             | in |
| 6 | c   |     |                 |    |                 |    |                 |    |
| 5 | a   |     |                 |    |                 |    |                 |    |
| 4 | b   | a   |                 |    |                 |    |                 |    |
| 3 | bc  | c   |                 |    |                 |    |                 |    |
| 2 | a   | b   |                 |    |                 |    |                 |    |
| 1 |     | a   |                 |    |                 |    |                 |    |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



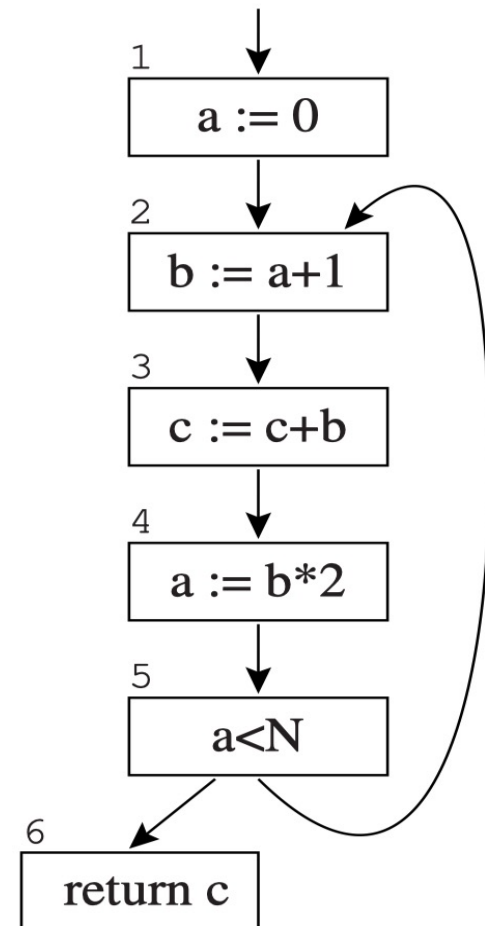
# Calculation of Liveness

- $out[n]$  is computed from  $in[s]$ .  $in[n]$  is computed from  $out[n]$
- We can speed the convergence by computing in **the opposite order** (from 6 to 1, from out to in)

|   | use | def | 1 <sup>st</sup> |    | 2 <sup>nd</sup> |    | 3 <sup>rd</sup> |    |
|---|-----|-----|-----------------|----|-----------------|----|-----------------|----|
|   |     |     | out             | in | out             | in | out             | in |
| 6 | c   |     |                 |    |                 |    |                 |    |
| 5 | a   |     |                 |    |                 |    |                 |    |
| 4 | b   | a   |                 |    |                 |    |                 |    |
| 3 | bc  | c   |                 |    |                 |    |                 |    |
| 2 | a   | b   |                 |    |                 |    |                 |    |
| 1 |     | a   |                 |    |                 |    |                 |    |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



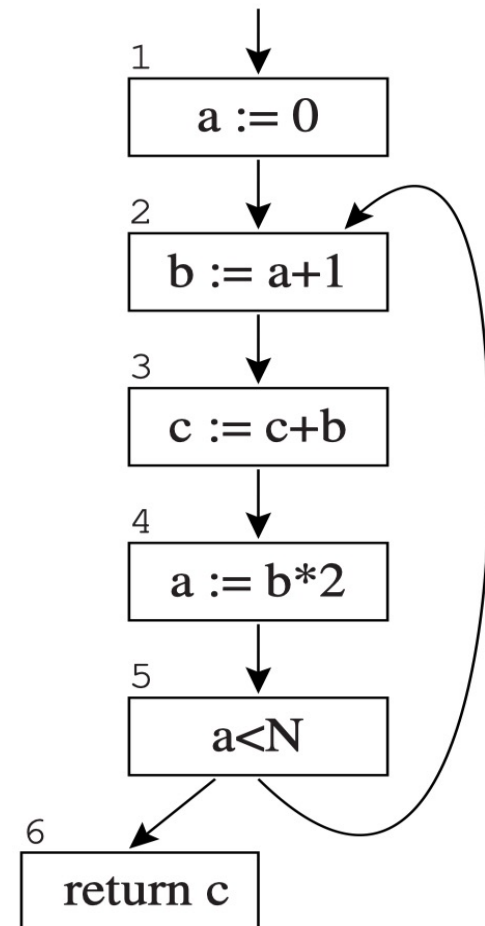
# Calculation of Liveness

- $out[n]$  is computed from  $in[s]$ .  $in[n]$  is computed from  $out[n]$
- We can speed the convergence by computing in **the opposite order** (from 6 to 1, from out to in)

|   | use | def | 1 <sup>st</sup> |    | 2 <sup>nd</sup> |    | 3 <sup>rd</sup> |    |
|---|-----|-----|-----------------|----|-----------------|----|-----------------|----|
|   |     |     | out             | in | out             | in | out             | in |
| 6 | c   |     |                 | c  |                 |    |                 |    |
| 5 | a   |     |                 |    |                 |    |                 |    |
| 4 | b   | a   |                 |    |                 |    |                 |    |
| 3 | bc  | c   |                 |    |                 |    |                 |    |
| 2 | a   | b   |                 |    |                 |    |                 |    |
| 1 |     | a   |                 |    |                 |    |                 |    |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



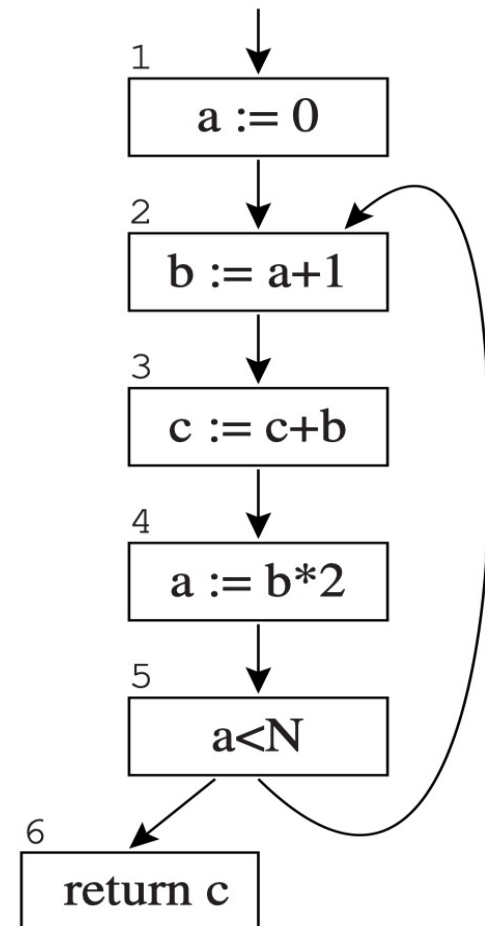
# Calculation of Liveness

- $out[n]$  is computed from  $in[s]$ .  $in[n]$  is computed from  $out[n]$
- We can speed the convergence by computing in **the opposite order** (from 6 to 1, from out to in)

|   | use | def | 1 <sup>st</sup> |    | 2 <sup>nd</sup> |    | 3 <sup>rd</sup> |    |
|---|-----|-----|-----------------|----|-----------------|----|-----------------|----|
|   |     |     | out             | in | out             | in | out             | in |
| 6 | c   |     |                 | c  |                 |    |                 |    |
| 5 | a   |     |                 |    |                 |    |                 |    |
| 4 | b   | a   |                 |    |                 |    |                 |    |
| 3 | bc  | c   |                 |    |                 |    |                 |    |
| 2 | a   | b   |                 |    |                 |    |                 |    |
| 1 |     | a   |                 |    |                 |    |                 |    |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



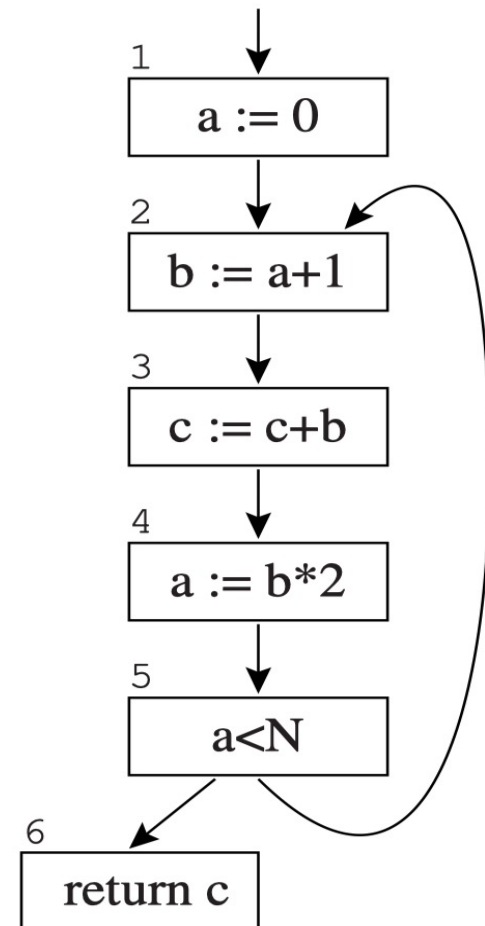
# Calculation of Liveness

- $out[n]$  is computed from  $in[s]$ .  $in[n]$  is computed from  $out[n]$
- We can speed the convergence by computing in **the opposite order** (from 6 to 1, from out to in)

|   | use | def | 1 <sup>st</sup> |    | 2 <sup>nd</sup> |    | 3 <sup>rd</sup> |    |
|---|-----|-----|-----------------|----|-----------------|----|-----------------|----|
|   |     |     | out             | in | out             | in | out             | in |
| 6 | c   |     |                 | c  |                 |    |                 |    |
| 5 | a   |     | c               |    |                 |    |                 |    |
| 4 | b   | a   |                 |    |                 |    |                 |    |
| 3 | bc  | c   |                 |    |                 |    |                 |    |
| 2 | a   | b   |                 |    |                 |    |                 |    |
| 1 |     | a   |                 |    |                 |    |                 |    |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



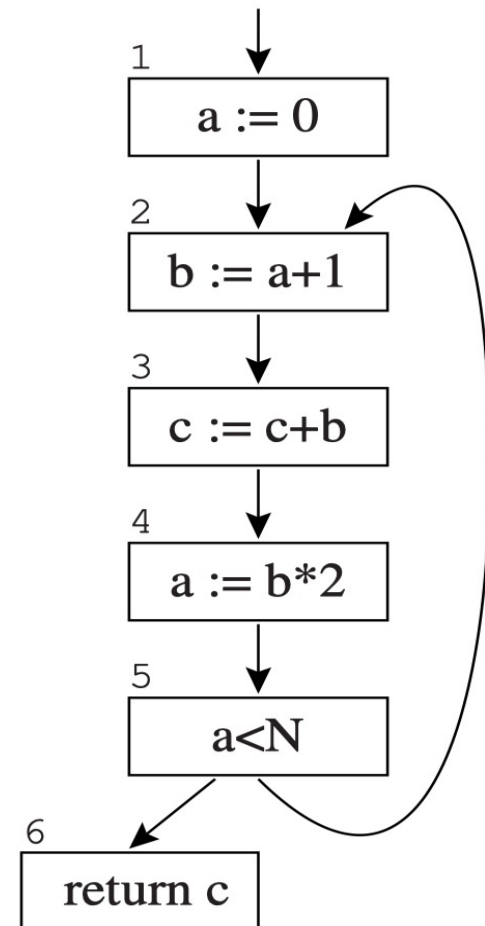
# Calculation of Liveness

- $out[n]$  is computed from  $in[s]$ .  $in[n]$  is computed from  $out[n]$
- We can speed the convergence by computing in **the opposite order** (from 6 to 1, from out to in)

|   | use | def | 1 <sup>st</sup> |    | 2 <sup>nd</sup> |    | 3 <sup>rd</sup> |    |
|---|-----|-----|-----------------|----|-----------------|----|-----------------|----|
|   |     |     | out             | in | out             | in | out             | in |
| 6 | c   |     |                 | c  |                 |    |                 |    |
| 5 | a   |     | c               | ac |                 |    |                 |    |
| 4 | b   | a   |                 |    |                 |    |                 |    |
| 3 | bc  | c   |                 |    |                 |    |                 |    |
| 2 | a   | b   |                 |    |                 |    |                 |    |
| 1 |     | a   |                 |    |                 |    |                 |    |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



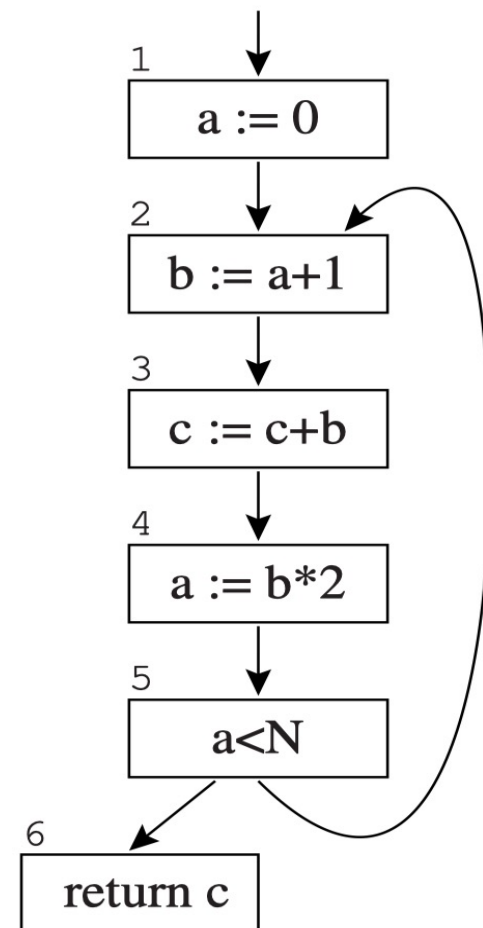
# Calculation of Liveness

- $out[n]$  is computed from  $in[s]$ .  $in[n]$  is computed from  $out[n]$
- We can speed the convergence by computing in **the opposite order** (from 6 to 1, from out to in)

|   | use | def | 1 <sup>st</sup> |    | 2 <sup>nd</sup> |    | 3 <sup>rd</sup> |    |
|---|-----|-----|-----------------|----|-----------------|----|-----------------|----|
|   |     |     | out             | in | out             | in | out             | in |
| 6 | c   |     |                 | c  |                 |    |                 |    |
| 5 | a   |     | c               | ac |                 |    |                 |    |
| 4 | b   | a   |                 |    |                 |    |                 |    |
| 3 | bc  | c   |                 |    |                 |    |                 |    |
| 2 | a   | b   |                 |    |                 |    |                 |    |
| 1 |     | a   |                 |    |                 |    |                 |    |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



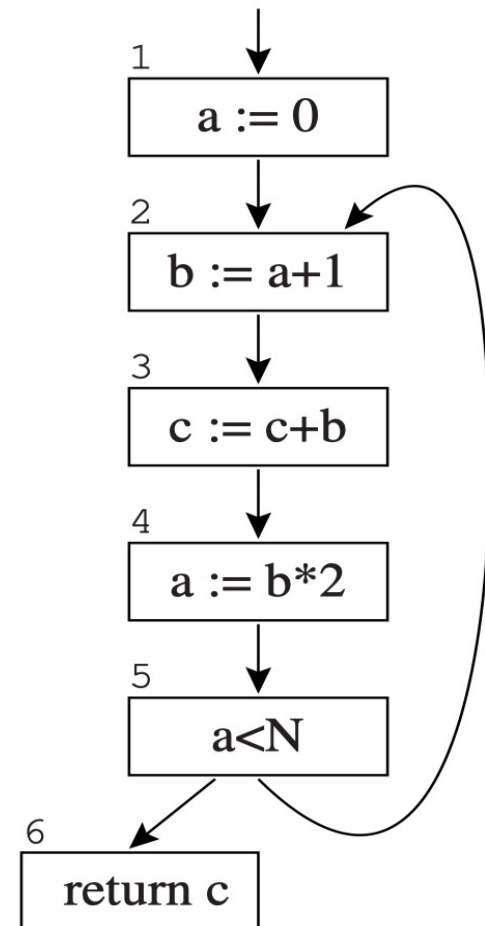
# Calculation of Liveness

- $out[n]$  is computed from  $in[s]$ .  $in[n]$  is computed from  $out[n]$
- We can speed the convergence by computing in **the opposite order** (from 6 to 1, from out to in)

|   | use | def | 1 <sup>st</sup> |    | 2 <sup>nd</sup> |    | 3 <sup>rd</sup> |    |
|---|-----|-----|-----------------|----|-----------------|----|-----------------|----|
|   |     |     | out             | in | out             | in | out             | in |
| 6 | c   |     |                 | c  |                 |    |                 |    |
| 5 | a   |     | c               | ac |                 |    |                 |    |
| 4 | b   | a   | ac              |    |                 |    |                 |    |
| 3 | bc  | c   |                 |    |                 |    |                 |    |
| 2 | a   | b   |                 |    |                 |    |                 |    |
| 1 |     | a   |                 |    |                 |    |                 |    |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



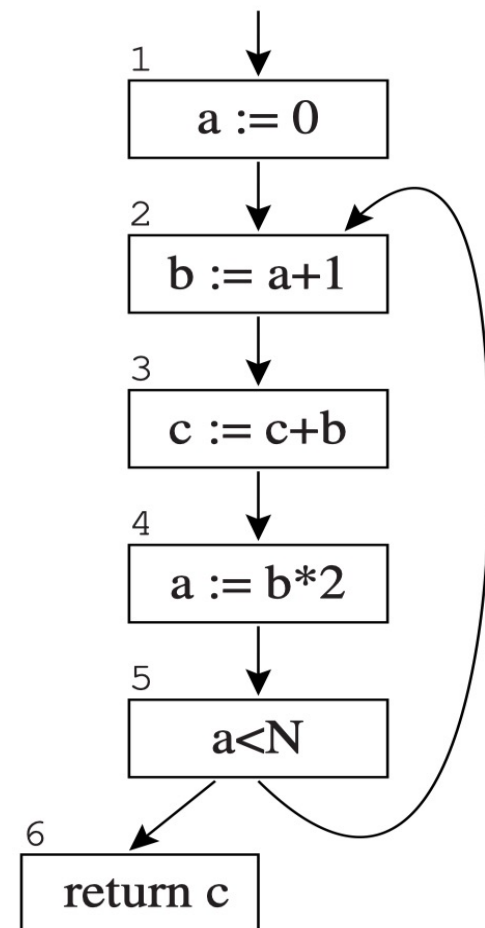
# Calculation of Liveness

- $out[n]$  is computed from  $in[s]$ .  $in[n]$  is computed from  $out[n]$
- We can speed the convergence by computing in **the opposite order** (from 6 to 1, from out to in)

|   | use | def | 1 <sup>st</sup> |    | 2 <sup>nd</sup> |    | 3 <sup>rd</sup> |    |
|---|-----|-----|-----------------|----|-----------------|----|-----------------|----|
|   |     |     | out             | in | out             | in | out             | in |
| 6 | c   |     |                 | c  |                 |    |                 |    |
| 5 | a   |     | c               | ac |                 |    |                 |    |
| 4 | b   | a   | ac              | bc |                 |    |                 |    |
| 3 | bc  | c   |                 |    |                 |    |                 |    |
| 2 | a   | b   |                 |    |                 |    |                 |    |
| 1 |     | a   |                 |    |                 |    |                 |    |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



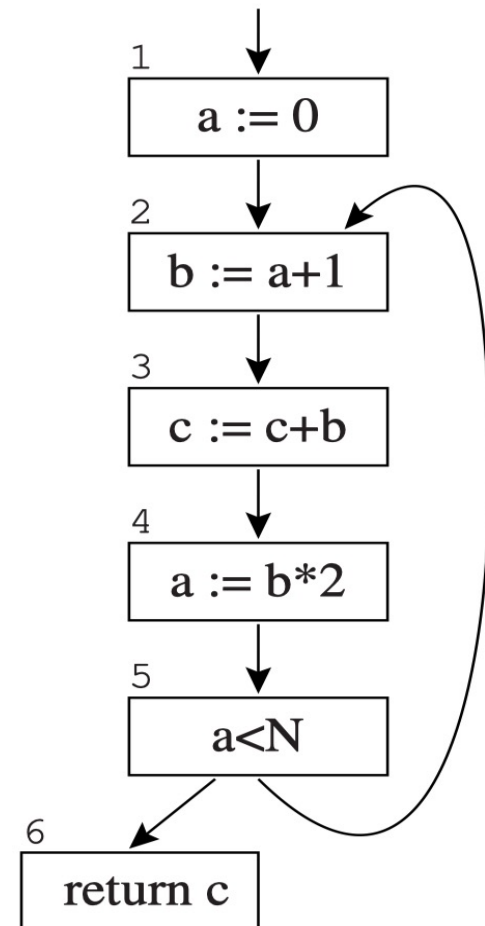
# Calculation of Liveness

- $out[n]$  is computed from  $in[s]$ .  $in[n]$  is computed from  $out[n]$
- We can speed the convergence by computing in **the opposite order** (from 6 to 1, from out to in)

|   |     |     | 1 <sup>st</sup> |    | 2 <sup>nd</sup> |    | 3 <sup>rd</sup> |    |
|---|-----|-----|-----------------|----|-----------------|----|-----------------|----|
|   | use | def | out             | in | out             | in | out             | in |
| 6 | c   |     |                 | c  |                 |    |                 |    |
| 5 | a   |     | c               | ac |                 |    |                 |    |
| 4 | b   | a   | ac              | bc |                 |    |                 |    |
| 3 | bc  | c   | bc              | bc |                 |    |                 |    |
| 2 | a   | b   | bc              | ac |                 |    |                 |    |
| 1 |     | a   | ac              | c  |                 |    |                 |    |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



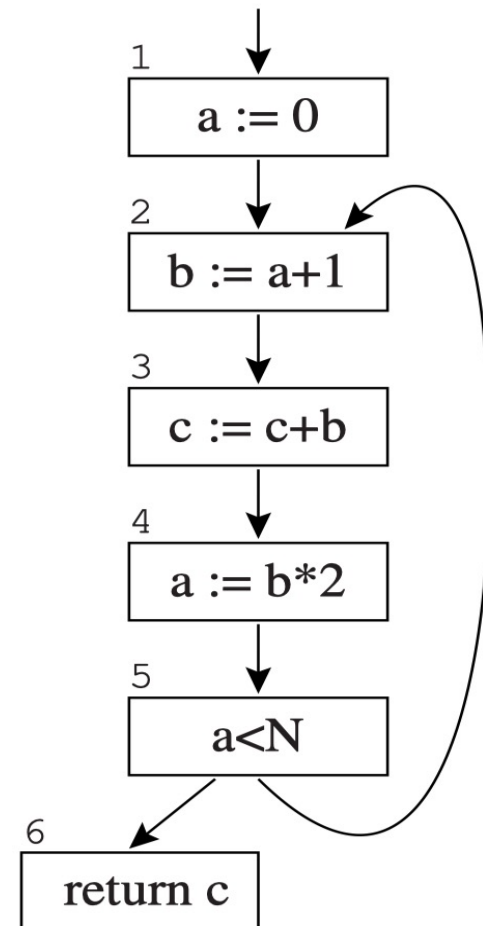
# Calculation of Liveness

- $out[n]$  is computed from  $in[s]$ .  $in[n]$  is computed from  $out[n]$
- We can speed the convergence by computing in **the opposite order** (from 6 to 1, from out to in)

|   |     |     | 1 <sup>st</sup> |    | 2 <sup>nd</sup> |    | 3 <sup>rd</sup> |    |
|---|-----|-----|-----------------|----|-----------------|----|-----------------|----|
|   | use | def | out             | in | out             | in | out             | in |
| 6 | c   |     |                 | c  |                 | c  |                 | c  |
| 5 | a   |     | c               | ac | ac              | ac | ac              | ac |
| 4 | b   | a   | ac              | bc | ac              | bc | ac              | bc |
| 3 | bc  | c   | bc              | bc | bc              | bc | bc              | bc |
| 2 | a   | b   | bc              | ac | bc              | ac | bc              | ac |
| 1 |     | a   | ac              | c  | ac              | c  | ac              | c  |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



# Calculation of Liveness

- When solving dataflow equations by iteration, the order of computation should follow the “flow”
- Liveness flows **backward** along control-flow arrows, and from “**out**” to “**in**”, so should the computation.

# Outline

- Liveness Analysis
  - Why, How, and What
  - Dataflow Equation for Liveness
  - Solving the Equations
- **More Discussions**
  - improvements
  - Implementation
  - Theoretical Results
- Interference Graphs
- Liveness in the Tiger Compiler

# Variants of the Calculation

- **Basic blocks:** Flow-graph nodes that have only one predecessor and one successor are not very interesting.
  - merging them with their predecessors and successors
  - obtaining a graph with fewer nodes, where each node represents a basic block
- **One variable at a time:** compute dataflow for one variable at a time as information about that variable is needed.
  - This is also practical, since many temporaries have very short live ranges.

# Representations of Sets

- How to represent **in[n]** and **out[n]** for implementation?
- Operations:
  - The union of sets
  - Subtraction between sets
- **Bit Arrays** (for dense set)
  - Suppose: **N** variables in the program, **K** bits per word
  - **N** bits for each set
  - The union of two sets: **or**-ing the corresponding bits at each position.
  - One set-union operation takes **N/K** operations.
- **Sorted Lists** (for sparse set)
  - sorted by any totally ordered key (such as variable name)
  - The union operation: merging the lists
- When the sets are sparse (fewer than **N/K** elements, on the average), the sorted-list representation is faster.

# Time Complexity

- **How fast is iterative dataflow analysis?**
- A program of size  $N$ : at most  $N$  nodes and at most  $N$  variables.
- Each **set-union operation** takes  $O(N)$  time.
- **For loop**: computes a constant number of set operations per node.  $O(N)$  nodes  $\Rightarrow O(N^2)$  time
- **Repeat loop**: The sum of the sizes of all **in** and **out** sets is  $2N^2$ , which is the most that the repeat loop can iterate
  - **Why?**
    - The **in** and **out** sets are monotonic, and cannot keep growing infinitely.
- Worst-case run time:
  - $O(N^4)$
- In practice :
  - **Between  $O(N)$  and  $O(N^2)$**
  - Proper computation order

```
for each n
  in[n] ← {}; out[n] ← {}
repeat
  for each n
    in'[n] ← in[n]; out'[n] ← out[n]
    in[n] ← use[n] ∪ (out[n] − def[n])
    out[n] ←  $\bigcup_{s \in \text{succ}[n]} \text{in}[s]$ 
  until in'[n] = in[n] and out'[n] = out[n] for all n
```

# Least Fixed Points

|   | X          |            | Y         |            | Z         |            |
|---|------------|------------|-----------|------------|-----------|------------|
|   | <i>use</i> | <i>def</i> | <i>in</i> | <i>out</i> | <i>in</i> | <i>out</i> |
| 1 |            | a          | c         | ac         | cd        | acd        |
| 2 | a          | b          | ac        | bc         | acd       | bcd        |
| 3 | bc         | c          | bc        | bc         | bcd       | bcd        |
| 4 | b          | a          | bc        | ac         | bcd       | acd        |
| 5 | a          |            | ac        | ac         | acd       | acd        |
| 6 | c          |            | c         |            | c         |            |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$

**TABLE 10.7.** X and Y are solutions to the liveness equations; Z is not a solution.

- Assume there is another program variable **d** not used in this fragment.
- Any solution to the dataflow equations is a conservation approximation.
  - If **a** is needed after node **n** in some execution, we can be assured that **a**  $\in out[n]$  in any solution of the equations.
  - But the converse is not true. **a**  $\in out[n]$  does not mean its value will really be used.

**Acceptable?**

# Least Fixed Points

- In the case of **liveness analysis**, if a variable is *thought to be live* then we will make sure to have its value in a register.
- A **conservative approximation** of liveness is that one may erroneously believe a variable is live, but will never erroneously believe it is dead.
- The consequence of a conservative approximation: the compiled code might use more registers than it really needs; but it will compute the right answer.

# Least Fixed Points

|   | X          |            | Y         |            | Z         |            |
|---|------------|------------|-----------|------------|-----------|------------|
|   | <i>use</i> | <i>def</i> | <i>in</i> | <i>out</i> | <i>in</i> | <i>out</i> |
| 1 |            | a          | c         | ac         | cd        | acd        |
| 2 | a          | b          | ac        | bc         | acd       | bcd        |
| 3 | bc         | c          | bc        | bc         | bcd       | bcd        |
| 4 | b          | a          | bc        | ac         | bcd       | acd        |
| 5 | a          |            | ac        | ac         | acd       | acd        |
| 6 | c          |            | c         |            | c         |            |

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$

**TABLE 10.7.** X and Y are solutions to the liveness equations; Z is not a solution.

- Consider the live-in sets Z, which fail to satisfy the dataflow equations.
  - Based on Z, b and c are never live at the same time
  - We can assign them to the same register
  - The resulting program would **compute the wrong answer**.

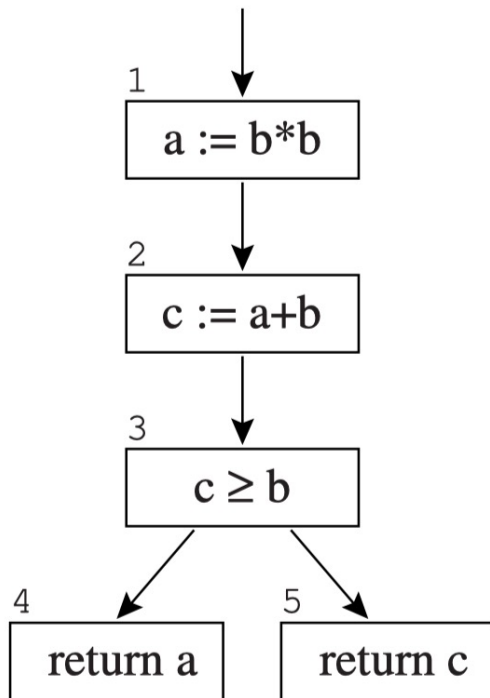
# Least Fixed Points

- A dataflow equation used for compiler optimization should be set up so that any solution to it provides conservative information to the optimizer
  - imprecise information may lead to suboptimal but never incorrect programs.

# Least Fixed Points

- **Theorem.** Equation 10.3 have more than one solutions
  - $X$  and  $Y$
- If  $X$  is the solution of Equation 10.3 and all solutions to Equation 10.3 contains Solution  $X$ , we say that  $X$  is the least solution (**least fixed point**) to Equation 10.3.
- Equation 10.3 have a least fixed point and the iteration algorithm described above always computes the least fixed point.
  - **Proof.** See Exercise 10.2.

# Static and Dynamic Liveness



- Node 1:  $a = b * b \geq 0$
- Node 2:  $c = a + b \geq b$
- Node 3: always be true
- Node 4 will never be reached

- The equations are ignorant of which way the conditional branch will go.
- “Smarter” equations would permit  $a$  and  $c$  to be assigned the same register.

# Static and Dynamic Liveness

- No compiler can ever fully understand how all the control flow in every program will work.
  - This is a fundamental mathematical theorem, derivable from the halting problem.
- **Theorem.** There is no program  $H$  that takes as input any program  $P$  and input  $X$  and (without infinite-looping) returns true if  $P(X)$  halts and false if  $P(X)$  infinite-loops.
- **Corollary.** No program  $H'(P, L)$  can tell, for any program  $P$  and label  $L$  in  $P$ , whether the label  $L$  is ever reached on an execution of  $P$ .
  - prove by showing that if  $H'$  exists, then  $H$  exists.
  - let  $L$  be the end of the program, `halt` => `goto L`

# Conservative Approximation

- This theorem does not mean that we can never tell if a given label is reached or not, just that there is not a **general** algorithm that can **always** tell.
  - We could improve our liveness analysis with some special-case algorithms.
  - But, no compiler can really tell if a variable's value is truly needed – whether the variable is truly live.
  - Instead, we have to make do with a conservative approximation.
    - Assume that any conditional branch goes both ways.

# Conservative Approximation

Thus, we have a dynamic condition and its static approximation:

- **Dynamic liveness:** a variable  $a$  is dynamically live at node  $n$  if **some execution** of the program goes from  $n$  to a use of  $a$  without going through any definition of  $a$ .
- **Static liveness:** a variable  $a$  is statically live at node  $n$  if there is **some path of control-flow edges** from  $n$  to some use of  $a$  that does not go through a definition of  $a$ .
- If  $a$  is dynamically live, it is also statically live.

# Outline

- Liveness Analysis
  - Why, How, and What
  - Dataflow Equation for Liveness
  - Solving the Equations
- More Discussions
- **Interference Graphs**
- Liveness in the Tiger Compiler

# Interference Graphs

- One of the most important applications of liveness analysis is for register allocation.
- We have a set of temporaries  $a, b, c, \dots$  and registers  $r1, \dots, rk$
- A condition that prevents  $a$  and  $b$  from being allocated to the same register is called **an interference**.
- Two types of interferences:
  - overlapping live ranges
  - when  $a$  must be generated by an instruction that cannot address register  $r1$ , then  $a$  and  $r1$  interfere

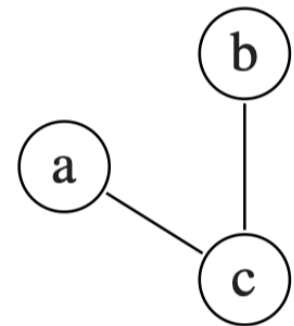
# Interference Graphs

The expression of interference:

- **a matrix**
  - **x** marking interferences
- **undirected graph**
  - node: each variable
  - edge: connecting variables that interfere

|   | a        | b        | c        |
|---|----------|----------|----------|
| a |          |          | <b>x</b> |
| b |          |          | <b>x</b> |
| c | <b>x</b> | <b>x</b> |          |

(a) Matrix



(b) Graph

# Special Treatment of MOVE instructions

- Do not create artificial interferences between the source and destination of a MOVE. Considering:

|                |            |
|----------------|------------|
| t := s         | (copy)     |
| ...            |            |
| x := ... s ... | (use of s) |
| ...            |            |
| y := ... t ... | (use of t) |

|                |            |
|----------------|------------|
| t := s         | (copy)     |
| t := ...       | (non-move) |
| x := ... s ... | (use of s) |
| ...            |            |
| y := ... t ... | (use of t) |

- Normally, we would make an interference edge  $(s, t)$ .
- But we do not need separate registers for  $s$  and  $t$ , since they contain the same value.
- Solution:** not to add an interference edge  $(s, t)$  in this case.
- If there is a later (nonmove) definition of  $t$  while  $s$  is still live, that will create the inference edge  $(t, s)$

# Interference Graphs

Therefore, the way to add interference edges for each new definition is:

- At any nonmove instruction  $n$  that defines a variable  $a$ , where  $\text{out}[n] = \{b_1, \dots, b_j\}$ , add interference edges  $(a, b_1), \dots, (a, b_j)$ .
- At a move instruction  $n$ , which is  $a := c$ , where  $\text{out}[n] = \{b_1, \dots, b_k\}$ , add interference edges  $(a, b_1), \dots, (a, b_k)$  for any  $b_i$  that is not the same as  $c$ .

# Liveness Analysis

What if a newly defined temporary is not live just after its definition?

- A variable is defined but never used.
  - No need to put it in a register
- But if the defining instruction is going to execute (perhaps for some other side effect), then it will write to some register.
  - That register had better not contain any other live variable.
  - Thus, zero-length live ranges do interfere with any live ranges that overlap them.

# Homework

- **10.1**

# Outline

- Liveness Analysis
  - Why, How, and What
  - Dataflow Equation for Liveness
  - Solving the Equations
- More Discussions
- Interference Graphs
- **Liveness in the Tiger Compiler**

# Liveness in The Tiger Compiler

The flow analysis for the Tiger compiler is done in two stages:

- The control flow of the **Assem** program is analyzed, producing a **control-flow graph**;
- The liveness of variables in the **control-flow graph** is analyzed, producing an **interference graph**.

# Graphs

```
/* graph.h: The graph datatype*/
typedef struct G_graph_ *G_graph;           /*The graph type*/
typedef struct G_node_ *G_node;             /*The node type*/
typedef struct G_nodeList_ *G_nodeList;     /*List of nodes*/
struct G_nodeList_ { G_node head; G_nodeList tail;};
G_graph G_Graph(void);                     /*Make a new graph*/
G_node G_Node(G_graph g, void *info);      /*Make new node in g*/
G_nodeList G_NodeList(G_node head, G_nodeList tail);
G_nodeList G_nodes(G_graph g);             /*Get the list of g's nodes*/
bool G_inNodeList(G_node a, G_nodeList l); /*Is a in l?*/
void G_addEdge(G_node from, G_node to);     /*Make a new edge*/
void G_rmEdge(G_node from, G_node to);      /*Delete an edge*/
void G_show(FILE *out, G_nodeList p, void showInfo(void *));
G_nodeList G_succ(G_node n);               /*Get n's successors*/
G_nodeList G_pred(G_node n);              /*Get n's predecessors*/
G_nodeList G_adj(G_node n);               /*G_succ(n) u G_pred(n)*/
bool G_goesTo(G_node a, G_node b);         /*Is there an edge from a to b?*/
int G_degree(G_node n);                   /*How many edges to/from n?*/
void *G_nodeInfo(G_node n);               /*Get n's info*/
typedef struct TAB_table_ *G_table;        /*Mapping nodes to whatever*/
G_table G_empty(void);                    /*Make a new table*/
void G_enter(G_table t, G_node n, void *v); /*make n->v in t*/
void *G_look(G_table t, G_node n);        /*find n->v, report v*/
```

# Graphs

- When using a graph in an algorithm, we want each node to represent something (e.g., an instruction in a program).
- To make node-to-thing mappings, we use a *G\_table* (a Hashtable).
- The following idiom associates information *x* with node *n* in a mapping mytable.

**G\_enter(mytable,n,x)**

- We can also put *x* directly inside *n*:
  - *n = G\_Node(g, x)*: creates a new node *n* with info *x*
  - *G\_nodeInfo(n)*: retrieves *x*

# Control-Flow Graphs

The FlowGraph package manages control-flow graphs:

- **node**: a instruction (or basic block)
- **edge(m, n)**: if instruction **m** can be followed by instruction **n** (either by a jump or by falling through)

# Control-Flow Graphs

```
/* flowgraph.h */  
Temp_tempList FG_def(G_node n);  
Temp_tempList FG_use(G_node n);  
bool FG_isMove(G_node n);  
  
G_graph FG_AssemFlowGraph(AS_instrList il);
```

A flow graph is just a **G\_graph** with some extra (hidden) information in each node. For each node:

- **FG\_def()**: what temporaries are defined at this node (destination registers of the instruction).
- **FG\_use()**: what temporaries are used at this node (source registers of the instruction).
- **FG\_isMove()**: whether this instruction is a MOVE instruction, one that could be deleted if the **def** and **use** were identical.

# Information Associated with the Nodes

- Information associated with the nodes
  - A flow graph: associate some **use** and **def** information with each node
  - The liveness-analysis algorithm: remember **live-in** and **live-out** information at each node
  - Other analyses on flow graphs: other kinds of information about each node
- We may not want to modify the data structure (which is a widely used interface) for each new analysis.
- A graph is a graph, and that a flow graph is a graph along with separately packaged auxiliary information (tables, or functions mapping nodes to whatever).

# Liveness Analysis

```
/* liveness.h */
typedef struct Live_moveList_ *Live_moveList;
struct Live_moveList_ {G_node src, dst; Live_moveList tail;};
Live_moveList Live_MoveList(G_node src, G_node dst, Live_moveList
tail);

struct Live_graph { G_graph graph; Live_moveList moves; };
Temp_temp Live_gtemp(G_node n);

struct Live_graph Live_liveness(G_graph flow);
```

- The **Liveness** module:
  - **input**: a flow graph
  - **output**: an interference graph + a list of node-pairs (representing MOVE instructions) that should be assigned the same register
- *Live\_gtemp* tells what temporary variable is represented by *n*.

# Liveness Analysis

- In the implementation of the **Liveness** module, it is useful to maintain a data structure that remembers what is live at the exit of each flow-graph node:

```
static void enterLiveMap(G_table t, G_node flowNode,  
Temp_tempList temps) {  
    G_enter(t, flowNode, temps);  
}
```

```
static Temp_tempList lookupLiveMap(G_table t, G_node  
flownode) {  
    return (Temp_tempList)G_look(t, flownode);  
}
```

- With a complete **liveMap**, we can construct a interference graph using the way we described just now.